Contents

Data Structures and Algorithms: Midterm Review Master Guide (v02) . . . 3
A First-Principles, Intuitive Reference for Bioinformaticians and Non-Java Programmers . . . 3
1. Overview & Design Philosophy . . . 3
2. Chapter Directory & Table of Contents . . . 3
3. Recommended Exam Study Pathway . . . 5
Chapter 0: Preface — Java Mental Models for Non-Programmers . . . 6
1. Why Bioinformaticians Need Data Structures and Algorithms . . . 6
2. Java vs. Python/R: The Mental Shift . . . 6
3. Physical Memory: Stack vs. Heap . . . 7
4. Primitives vs. References: Cash vs. Warehouse Address . . . 8
5. Minimal Java Syntax Survival Kit . . . 9
6. How to Read Java Code Without Getting Lost . . . 10
Chapter 1: Algorithm Analysis, Asymptotics, and Growth Rates . . . 11
1. The Core Question: Why Can't We Just Use a Stopwatch? . . . 11
2. What is Input Size n? Biological Context . . . 11
3. The Hierarchy of Growth Rates: From Instant to Impossible . . . 11
4. Asymptotic Notations: Big-O, Big-Ω, and Big-Θ . . . 13
5. How to Analyze Code Loops: Step-by-Step Rules . . . 14
6. Real Example: Maximum Subarray Sum (The 3 Generations) . . . 15
Chapter 2: Arrays and Memory Contiguity . . . 17
1. What is an Array? The Contiguous Memory Model . . . 17
2. The Array Trade-Off: Instant Access vs. Expensive Shifts . . . 17
3. Static Array (`int[]`) vs. Dynamic Array (`ArrayList`) . . . 18
4. High-Yield Array Templates for the Midterm . . . 18
Chapter 3: Searching Algorithms and Sliding Windows . . . 21
1. Searching: How to Find What You Need . . . 21
2. Linear Search and Sentinel Optimization . . . 21
3. Binary Search: The Logarithmic Powerhouse . . . 22
4. Exam Variations: First and Last Occurrences . . . 24
5. The Sliding Window Technique . . . 25
Chapter 4: Elementary Sorting Algorithms (Bubble, Selection, Insertion) . . . 27
1. Why Sorting Matters & The Concept of Stability . . . 27
2. Bubble Sort: Adjacent Pairwise Swaps . . . 27
3. Selection Sort: Finding Extremes with Minimum Swaps . . . 29
4. Insertion Sort: The Card Player's Algorithm . . . 30
5. Master Comparison: Elementary Sorting Algorithms . . . 32
Chapter 5: Stacks (LIFO), Applications, and Monotonic Patterns . . . 33
1. The Stack ADT: Last-In, First-Out (LIFO) . . . 33
2. Implementations: Array Stack vs. Linked Stack . . . 33
3. High-Yield Exam Applications . . . 34
Chapter 6: Queues (FIFO), Circular Queues, Deques, and Priority Queues . . . 39
1. The Queue ADT: First-In, First-Out (FIFO) . . . 39
2. The Naive Array Queue Flaw: The $O(n)$ Dequeue Penalty . . . 39

3. The Circular Array Queue (Ring Buffer) 40
4. Linked Queue: Dynamic Memory with Two Pointers 42
5. High-Yield Exam Applications 43
6. Priority Queues & Heaps: Priority Over Arrival 44
Chapter 7: Linked Lists in Depth: Nodes, Pointers, and Invariants 46
1. Why Linked Lists? The Non-Contiguous Memory Model 46
2. Array vs. Linked List: The Fundamental Trade-Offs 47
3. Singly Linked List Mechanics 47
4. The Sentinel (Dummy Head) Node: Eliminating Edge Cases 48
5. Doubly Linked Lists: Two-Way Navigation 49
6. High-Yield Exam Templates 49
Chapter 8: Recursion & The Call-Stack Mechanics 53
1. The Recursive Mental Model: Self-Similarity and Contracts 53
2. The Three Mandatory Rules of Recursion 53
3. Under the Hood: The JVM Call Stack 54
4. Classic Recursive Examples & Walkthroughs 54
5. Towers of Hanoi: The Masterclass in Recursion 56
Chapter 9: Merge Sort: The Divide-and-Conquer Masterclass 58
1. The Divide-and-Conquer Paradigm 58
2. The Core Merging Engine: Interleaving Two Sorted Halves in $O(n)$ 58
3. Merge Implementations: Standard vs. Sentinel Method 59
4. The Complete Merge Sort Method 60
5. Visualizing the Recursion Tree 61
6. Mathematical Analysis & Invariants 62
Chapter 10: Midterm Synthesis, Master Tables, and Decision Trees 63
1. Master Complexity Reference Tables 63
2. The 10 Classic Midterm Traps Exposed 65
3. Algorithmic Decision Tree: What Should You Use? 67
4. Midterm Tracing Templates 67

Data Structures and Algorithms: Midterm Review Master Guide (v02)

A First-Principles, Intuitive Reference for Bioinformaticians and Non-Java Programmers

Covering Curriculum from Week 00 through Week 06 (Recursion & Merge Sort)

---

1. Overview & Design Philosophy

This guide is tailored specifically for bioinformaticians, computational biologists, and researchers coming from Python/R who need a clear, rigorous understanding of Data Structures and Algorithms (DSA) without getting bogged down in low-level Java boilerplate.

Rather than presenting dry mathematical proofs or language-specific trivia, this v02 review book emphasizes: 1. **Physical Computer Memory Models:** Understanding the difference between contiguous RAM blocks (Arrays) and scattered Heap objects (Linked Lists, Nodes, Objects). 2. **Computational Scaling:** Why algorithm complexity determines whether an analysis on a 3-billion-base human genome completes in 3 seconds ($O(n)$) or takes centuries ($O(n^2)$). 3. **Biological Analogies:** Mapping abstract concepts directly to sequencing reads, chromosome coordinates, RNA secondary structures, and alignment pipelines. 4. **Copy-Pasteable Exam Templates:** Minimal, straightforward Java templates without confusing generics or unnecessary object-oriented hierarchies. 5. **How Each Algorithm Works:** Step-by-step mechanical explanations and hand-tracing walkthroughs for exam questions.

---

2. Chapter Directory & Table of Contents

The complete handbook is divided into modular, focused chapter documents:

Part / Chapter	Document Link	Core Topics & Focus Areas
Chapter 0	chapter-00_preface-java-mental-models.md	Java vs. Python/R, Stack (workbench) vs. Heap (storage), Primitive vs. Reference semantics (“Cash vs. Warehouse Address”), Minimal Java syntax survival kit.
Chapter 1	chapter-01_algorithm-analysis-and-asymptotics.md	Why wall-clock time lies (RAM model), Growth hierarchy ($O(1)$ to $O(2^n)$), Big-O / Omega / Theta (Ceiling, Floor, Tight Bound), How to analyze code loops, Maximum Subarray Sum (3 generations).
Chapter 2	chapter-02_arrays-and-memory-contiguity.md	Contiguous RAM layout (96-well plate metaphor), $O(1)$ address formula, Array shifting penalties on insert/delete, Static vs. Dynamic arrays (`ArrayList`), High-yield array templates.
Chapter 3	chapter-03_searching-and-sliding-windows.md	Linear search vs. Sentinel linear search, Binary search step-by-step mechanics, Safe midpoint calculation without overflow, First and last occurrence variants, Fixed-width sliding windows ($O(1)$ update).
Chapter 4	chapter-04_elementary-sorting-algorithms.md	Stability concept & why it matters to data integrity, Bubble Sort (early-exit optimization), Selection Sort (min-to-front vs. max-to-end; why it is unstable), Insertion Sort (card player’s algorithm; ascending vs. descending)
Chapter 10	chapter-10_[illegible]-trees.md	Master sorting comparison table, [illegible]

3. Recommended Exam Study Pathway

```
flowchart LR
    Step1["Step 1: Core Mental Models\nChapter 0 & Chapter 1\nStack/Heap & Big-O"] --> Step2["Step 2: Linear Collections\nChapters 2, 5, 6, 7\nArrays, Stacks, Queues, Lists"]
    Step2 --> Step3["Step 3: Algorithms & Trees\nChapters 3, 4, 8, 9\nSearch, Sort, Recursion, Merge"]
    Step3 --> Step4["Step 4: Midterm Synthesis\nChapter 10\nDecision Trees & Traps"]
```

1. **Quick Conceptual Alignment:** Start with Chapter 10 to review the master complexity tables and the 10 classic midterm traps.
2. **Algorithm Mechanics & Tracing:** Review Chapter 4 and Chapter 9 for step-by-step sorting traces.
3. **Pointers & Data Structures:** Review Chapter 5, Chapter 6, and Chapter 7 for stack/queue algorithms, circular buffer math, and linked list pointer splicing.

Chapter 0: Preface — Java Mental Models for Non-Programmers

1. Why Bioinformaticians Need Data Structures and Algorithms

In bioinformatics and computational biology, daily work involves processing massive volumes of biological data: - Hundreds of millions of DNA/RNA reads in FASTQ files. - Hundreds of thousands of gene expression profiles across single cells. - Millions of genomic variants mapped along chromosome coordinates. - Thousands of 3D protein structures with atomic spatial coordinates.

When writing high-level scripts in Python or R using packages like `pandas`, `Biopython`, or `Seurat`, the computer hides memory management and low-level loops behind clean functions. However, if an algorithm uses a naive nested loop ($O(n^2)$) instead of an indexed lookup ($O(\log n)$ or $O(1)$), a job that should finish in 5 seconds will run for 3 weeks or crash your server with an Out-Of-Memory (OOM) error.

Data Structures and Algorithms (DSA) is simply the study of: 1. **How data is organized in physical computer memory** (Data Structures). 2. **How step-by-step procedures manipulate that data efficiently** (Algorithms).

---

2. Java vs. Python/R: The Mental Shift

Coming from Python or R, Java feels strict and verbose. Understanding why Java behaves this way makes it much easier to write and read.

```
flowchart TD
    subgraph PythonR["Python / R: Dynamic & Relaxed"]
        P1["x = 10"] --> P2["Interpreter decides type at runtime"]
        P2 --> P3["x = 'ATCG' (Type can freely change)"]
        P3 --> P4["Flexible, but slower execution and hidden memory
overhead"]
    end

    subgraph JavaLang["Java: Static & Explicit"]
        J1["int x = 10;"] --> J2["Compiler fixes type and memory size
upfront"]
        J2 --> J3["x = 'ATCG' is an immediate compile-time error"]
        J3 --> J4["Rigid, but executes near C speed with predictable
memory"]
    end
```

The Test Tube Metaphor

- In **Python/R**, variables are like unlabeled plastic bags. You can put an integer in a bag today, replace it with a text string tomorrow, and append 100 new items whenever you want.

- In **Java**, variables are like calibrated glass laboratory test tubes. When you declare `int x;`, you manufacture a tube specifically molded to hold exactly one 32-bit integer. You cannot put a decimal number or text string into that tube.

Key Syntax Differences at a Glance

Feature	Python / R	Java	Why Java Does This
Variable Declaration	`x = 5`	`int x = 5;`	Memory must be pre-allocated before running.
Code Blocks	Indentation (whitespace)	Curly braces `{ }`	Explicit boundaries; whitespace does not change logic.
Statement End	Newline	Semicolon `;`	Tells the compiler exactly where an instruction terminates.
Array Sizing	Dynamic (`list.append(x)`)	Fixed at creation (`new int[10]`)	Arrays occupy contiguous physical memory slots.
Method Wrapping	Free functions anywhere	Every function lives inside a `class`	Enforces modular encapsulation.

---

3. Physical Memory: Stack vs. Heap

To write correct code and understand pointers/references, you only need one simple mental model of computer memory: the **Workbench (Stack)** and the **Central Cold Storage (Heap)**.

```
flowchart LR
    subgraph StackMemory["Call Stack (The Workbench)"]
        direction TB
        F1["Current Method Frame: main()"]
        V1["Primitive: int count = 5"]
        V2["Reference: int[] arr = 0x7A40"]
        F1 --- V1
        F1 --- V2
    end

    subgraph HeapMemory["Heap (Central Storage Freezer)"]
        direction TB
        Obj1["Array Object at 0x7A40\n[ 10, 20, 30, 40, 50 ]"]
```

```
    end

    V2 -->|Holds memory address 0x7A40| Obj1
```

1. The Call Stack (The Lab Workbench)
- The Stack stores variables needed for the method currently executing.
- It is fast, organized, and temporary.
- When a method is called, its workbench (a **Stack Frame**) is created. When the method finishes (`return`), its entire workbench is cleared instantly.

2. The Heap (The Central Cold Storage)
- The Heap stores large objects, linked list nodes, and all arrays (`new int[100]`).
- It is large, flexible, and shared across the entire program.
- Objects stay alive in the Heap until no variable on any workbench points to them anymore (then the Java **Garbage Collector** safely recycles them).

---

4. Primitives vs. References: Cash vs. Warehouse Address

This is the single most tested concept in beginner Java exams.

```
flowchart TD
    subgraph Primitives["Primitive Types (Cash in Wallet)"]
        P["int a = 10;\nint b = a;\nb = 20;"]
        P_Note["a is still 10!\nCopying a primitive duplicates the actual value."]
    end

    subgraph References["Reference Types (Warehouse Coordinate)"]
        R["int[] a = {1, 2, 3};\nint[] b = a;\nb[0] = 99;"]
        R_Note["a[0] is now 99!\nb holds a copy of the address, pointing to the same array."]
    end
```

1. Primitive Variables (Direct Value)
- Types: `int`, `double`, `boolean`, `char`, `long`.
- Stored directly on the Stack.
- **Analogy:** Carrying $10 in cash in your wallet. If you give a copy of that $10 bill to a colleague, and they spend it, your $10 remains in your wallet.

```java
int a = 10;
int b = a;  // b gets a copy of the number 10
b = 20;     // Modifying b has ZERO effect on a
System.out.println(a); // Prints 10
```

2. Reference Variables (Memory Address)
- Types: All arrays (`int[]`, `String[]`), all objects (`Node`, `Scanner`, custom classes).

- The variable on the Stack does **not** hold the data. It holds a 64-bit hexadecimal number representing the **physical address** where the object sits in the Heap.
- **Analogy:** Writing the freezer coordinate `"Shelf 4, Box B"` on an index card. If you give a copy of the index card to your lab partner, both of you are looking at the exact same box. If your partner swaps a sample tube inside that box, you will see the change immediately.

```
int[] a = new int[]{10, 20, 30};
int[] b = a;  // b copies the ADDRESS, not the array! Both point to the
same array.
b[0] = 99;    // We change slot 0 through reference b

System.out.println(a[0]); // Prints 99! a and b share the exact same Heap
object.
```

---

5. Minimal Java Syntax Survival Kit

For exam writing, you only need a handful of core syntax structures. Keep them simple and avoid exotic features.

1. Minimal Program Skeleton

Every exam question usually requires writing a method or a standalone class:

```
public class Solution {
    // Methods are written here
    public static void main(String[] args) {
        // Test code goes here
    }
}
```

2. Arrays: Creation and Access

```
// Option A: Declare and allocate empty array of size 5 (defaults to zeros)
int[] numbers = new int[5];

// Option B: Declare with literal values
int[] values = {10, 25, 40, 55, 70};

// Length of array (NOTE: .length has NO parentheses for arrays)
int size = values.length;

// Accessing slots (indices run from 0 to size - 1)
int first = values[0];
int last = values[values.length - 1];
```

3. Loops: The Workhorses

```java
// Standard for-loop (use when you need the index)
for (int i = 0; i < values.length; i++) {
    System.out.println("Index " + i + " contains: " + values[i]);
}

// While-loop (use when termination depends on a condition, e.g., pointers)
int i = 0;
while (i < values.length) {
    // do work
    i++;
}
```

4. Swapping Two Elements in an Array

You will write this pattern repeatedly in Bubble Sort, Selection Sort, and Quicksort:

```java
// Swapping values[i] and values[j] requires a temporary variable
int temp = values[i];
values[i] = values[j];
values[j] = temp;
```

---

6. How to Read Java Code Without Getting Lost

When inspecting Java code on an exam paper: 1. **Ignore the Class Shell:** Look directly for the method name, its return type, and its input parameters: `public static int search(int[] arr, int target)` - Takes an integer array `arr` and an integer `target`. - Must return a single integer (`int`). 2. **Track Local Variables:** Draw a quick table on scratch paper with columns for each variable (`i`, `j`, `temp`, `swapped`). 3. **Follow Loop Conditions Step-by-Step:** Check initial values, loop boundary checks (`i < n`), and increment steps (`i++`).

Chapter 1: Algorithm Analysis, Asymptotics, and Growth Rates

1. The Core Question: Why Can't We Just Use a Stopwatch?

When comparing two algorithms in bioinformatics or computer science, why can't we simply run both on our computer and time them using `System.currentTimeMillis()`?

Wall-clock time is misleading because it depends on factors completely unrelated to the algorithm: 1. **Hardware differences:** Running on a 64-core compute cluster vs. an old laptop. 2. **Background operating system load:** Active web browsers, indexing tasks, or other users sharing the server. 3. **Compiler and runtime optimizations:** Just-In-Time (JIT) compilation differences. 4. **Input peculiarities:** Whether the specific test file happens to be small or already sorted.

To compare algorithms objectively, computer scientists use the **Random Access Machine (RAM) Model**: - We count the **number of fundamental steps** executed as a function of the **input size** n. - A fundamental step is an operation that takes constant time ($O(1)$): - Basic math (`+` , `-` , `*` , `/`) - Comparisons (`<` , `>` , `==`) - Variable assignments (`x = y`) - Direct array indexing (`arr[i]`)

---

2. What is Input Size n? Biological Context

The variable n represents the amount of data your program must process:

Biological Dataset	Typical Input Size n
Single PCR Primer or Oligo	$n \approx 20 \text{ to } 30 \text{ base pairs}$
Protein Sequence (e.g., Hemoglobin)	$n \approx 300 \text{ to } 500 \text{ amino acids}$
Human Protein-Coding Genes	$n \approx 20,000 \text{ genes}$
Bacterial Genome (*E. coli*)	$n \approx 4.6 \times 10^6 \text{ base pairs}$
Human Genome (Haploid)	$n \approx 3.1 \times 10^9 \text{ base pairs}$
High-Throughput Sequencing Run	$n \approx 10^8 \text{ to } 10^9 \text{ short reads}$

As n grows from 100 to 1,000,000, the efficiency of an algorithm determines whether your script finishes in milliseconds or runs past your graduation.

---

3. The Hierarchy of Growth Rates: From Instant to Impossible

Below is the standard hierarchy of time complexities, ranked from fastest to slowest:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

```
flowchart LR
    A["O(1)\nConstant"] --> B["O(log n)\nLogarithmic"]
```

```
    B --> C["O(n)\nLinear"]
    C --> D["O(n log n)\nLinearithmic"]
    D --> E["O(n^2)\nQuadratic"]
    E --> F["O(2^n)\nExponential"]
```

Intuitive Explanations

1. $O(1)$ **— Constant Time:**
   - **Meaning:** Takes the exact same time regardless of whether $n = 10$ or $n = 10,000,000$.
   - **Analogy:** Grabbing the first tube from a test tube rack. Having 10 tubes or 1,000 tubes in the rack doesn't change how fast your hand reaches slot 0.
   - **Example:** `int first = arr[0];`
2. $O(\log n)$ **— Logarithmic Time:**
   - **Meaning:** Every single step cuts the remaining problem size in half.
   - **Analogy:** Looking up a species name in an alphabetical biological encyclopedia. You open to the middle, see if your target is before or after, and discard half the book.
   - **Scaling Power:** For a human genome of 3 billion bases, $\log_2(3 \times 10^9) \approx 32$ steps!
3. $O(n)$ **— Linear Time:**
   - **Meaning:** Running time is directly proportional to dataset size. If data doubles, time doubles.
   - **Analogy:** Pipetting a reagent into every single well of an n-well plate.
   - **Example:** Scanning a DNA sequence end-to-end to count total Guanine bases.
4. $O(n \log n)$ **— Linearithmic Time:**
   - **Meaning:** The gold standard for sorting. It divides data recursively ($\log n$ levels) and does linear work (n) across each level.
   - **Analogy:** Merge Sort dividing sequence reads into halves and merging them back in order. Fast enough to handle millions of reads in seconds.
5. $O(n^2)$ **— Quadratic Time:**
   - **Meaning:** A nested loop where every item must be compared against every other item. If data grows by $10\times$, runtime increases by $100\times$!
   - **Analogy:** Building an all-against-all pairwise distance matrix between n patient genomes without indexing. Fine for $n = 100$, but catastrophic for $n = 100,000$.
6. $O(2^n)$ **— Exponential Time:**
   - **Meaning:** Doubling the input size squares the runtime.
   - **Analogy:** Trying every possible combination of n molecular switches. For $n = 50$, the operations exceed the number of seconds since the Big Bang.

Time Comparison on Real Hardware (1 Billion ops/sec)

Complexity	$n = 100$	$n = 10,000$	$n = 10^6$ (1 Million)	$n = 10^9$ (1 Billion)
$O(1)$	1 ns	1 ns	1 ns	1 ns
$O(\log_2 n)$	7 ns	14 ns	20 ns	30 ns
$O(n)$	$0.1\ \mu$ s	$10\ \mu$ s	1 ms	1 second
$O(n \log_2 n)$	$0.7\ \mu$ s	$140\ \mu$ s	20 ms	30 seconds
$O(n^2)$	$10\ \mu$ s	100 ms	16.7 minutes	≈ 31.7 years
$O(2^n)$	4×10^{13} years	Impossible	Impossible	Impossible

---

4. Asymptotic Notations: Big-O, Big-Ω, and Big-Θ

Exam questions frequently ask you to distinguish between Big-O, Big-Omega, and Big-Theta. Remember them as Ceiling, Floor, and Tight Envelope:

```
flowchart TD
    subgraph Asymptotics["Asymptotic Boundaries"]
        BO["Big-O: O(g(n))\nUpper Bound (Ceiling)\nWorst-case guarantee"]
        BT["Big-Theta: Theta(g(n))\nTight Bound (Exact Envelope)\nUpper and
Lower match"]
        BOM["Big-Omega: Omega(g(n))\nLower Bound (Floor)\nBest-case
baseline"]
    end
```

1. Big-O (O): The Upper Bound ("Ceiling")

- **Definition:** The algorithm will not grow faster than this rate as n grows large.
- **Plain English:** *"Worst case, it will take at most this much time."*
- **Mathematical Form:** $T(n) \leq c \cdot g(n)$ for large n.
- **Rule of Thumb:** Drop lower-order terms and constant coefficients.
  - If $T(n) = 3n^2 + 50n + 1000$, the dominant term is n^2. Thus, $T(n) = O(n^2)$.

2. Big-Omega (Ω): The Lower Bound ("Floor")

- **Definition:** The algorithm will require at least this much work.
- **Plain English:** *"Best case, it cannot do better than this baseline."*
- **Mathematical Form:** $T(n) \geq c \cdot g(n)$ for large n.
- **Example:** Finding an element in an unsorted array of size n requires checking at least 1 element in the best case ($\Omega(1)$), but any algorithm that checks the entire array has a lower bound of $\Omega(n)$.

3. Big-Theta (Θ): The Tight Bound ("Envelope")

- **Definition:** When an algorithm's upper bound and lower bound match, it belongs to $\Theta(g(n))$.

- **Plain English:** *"The algorithm grows at the exact same rate as $g(n)$."*
- **Condition:** $T(n) = \Theta(g(n))$ if and only if $T(n) = O(g(n))$ AND $T(n) = \Omega(g(n))$.

---

5. How to Analyze Code Loops: Step-by-Step Rules

On exams, you will be given Java code snippets and asked to state the time complexity. Follow these 4 standard patterns:

Pattern 1: Single Linear Loop

```
for (int i = 0; i < n; i++) {
    // 1 constant operation
    count++;
}
```

- **Analysis:** The loop runs n times. Each pass takes $O(1)$. Total = $n \times O(1) = O(n)$.

Pattern 2: Nested Independent Loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        matrix[i][j] = 0;
    }
}
```

- **Analysis:** Outer loop runs n times. For each outer step, the inner loop runs n times. Total = $n \times n = O(n^2)$.

Pattern 3: Triangular Nested Loop

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        compare(arr[i], arr[j]);
    }
}
```

- **Analysis:**
  - When $i = 0$, inner loop runs $n - 1$ times.
  - When $i = 1$, inner loop runs $n - 2$ times.
  - When $i = n - 2$, inner loop runs 1 time.
  - Total steps = $(n-1) + (n-2) + ... + 1 = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$.
  - Dropping lower terms and constants: $\Theta(n^2)$.

Pattern 4: Halving / Doubling Loops (Logarithmic)

```
int i = 1;
while (i < n) {
    i = i * 2; // Index doubles every step!
}
```

- **Analysis:** i takes values: $1, 2, 4, 8, 16, ..., 2^k$.
- We stop when $2^k \geq n \implies k = \log_2 n$.
- Total runtime = $O(\log n)$.

---

6. Real Example: Maximum Subarray Sum (The 3 Generations)

Given an array of integers (which can include negative numbers), find the contiguous subarray with the largest sum. Example: `arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`. The answer is `[4, -1, 2, 1]` with sum = `6`.

Generation 1: Naive Cubic $O(n^3)$

Checks all possible starting points i, ending points j, and adds every number between them.

```
public static int maxSubarraySumNaive(int[] arr) {
    int n = arr.length;
    int maxSum = Integer.MIN_VALUE;

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int currentSum = 0;
            for (int k = i; k <= j; k++) { // 3rd nested loop!
                currentSum += arr[k];
            }
            if (currentSum > maxSum) {
                maxSum = currentSum;
            }
        }
    }
    return maxSum;
}
```

- **Complexity:** 3 nested loops $\implies O(n^3)$. Extremely slow.

Generation 2: Quadratic $O(n^2)$

Reuses the running sum instead of recounting from scratch.

```
public static int maxSubarraySumQuadratic(int[] arr) {
    int n = arr.length;
    int maxSum = Integer.MIN_VALUE;

    for (int i = 0; i < n; i++) {
        int currentSum = 0;
        for (int j = i; j < n; j++) {
            currentSum += arr[j]; // Just add the new element
            if (currentSum > maxSum) {
                maxSum = currentSum;
            }
        }
```

```
    }
    return maxSum;
}
```

- **Complexity:** 2 nested loops $\implies O(n^2)$. Better, but still too slow for large n.

Generation 3: Kadane's Algorithm $O(n)$ (Optimal)
The Core Insight: As you walk through the array, at each number you ask: *"Should I add this number to my existing running streak, or should I throw away the past and start a fresh streak with just this number?"*

```
public static int maxSubarraySumKadane(int[] arr) {
    int maxSoFar = arr[0];
    int currentStreak = arr[0];

    for (int i = 1; i < arr.length; i++) {
        // Either continue current streak or start fresh at arr[i]
        currentStreak = Math.max(arr[i], currentStreak + arr[i]);
        // Update the best sum seen so far
        maxSoFar = Math.max(maxSoFar, currentStreak);
    }
    return maxSoFar;
}
```

- **Complexity:** Single loop through array $\implies \Theta(n)$ time, $O(1)$ space.

Chapter 2: Arrays and Memory Contiguity

1. What is an Array? The Contiguous Memory Model

An **array** is the most fundamental data structure in computing. In Java, an array is a collection of elements of the same type stored in **contiguous (strictly adjacent) physical memory slots**.

```
flowchart LR
    subgraph RAM["Contiguous Physical RAM Slots"]
        A0["Index 0\n[ 42 ]\nAddress: 1000"] --- A1["Index
1\n[ 15 ]\nAddress: 1004"]
        A1 --- A2["Index 2\n[ 88 ]\nAddress: 1008"]
        A2 --- A3["Index 3\n[ 23 ]\nAddress: 1012"]
    end
```

The 96-Well Microplate Metaphor

Imagine a plastic 96-well PCR plate or pill organizer. - Every well is the exact same size (4 bytes for an `int`). - All wells are welded together in a fixed, continuous row. - If you know where well 0 is located, you don't have to count through all the wells to find well 50; you simply jump directly to it:

$$\text{Address}(\text{arr}[i]) = \text{Base Address} + (i \times \text{Size of Element})$$

Because multiplication and addition take 1 step, **accessing or modifying an element at any index `arr[i]` takes instantaneous $O(1)$ constant time**.

---

2. The Array Trade-Off: Instant Access vs. Expensive Shifts

While looking up `arr[i]` is instantaneous, changing the size or inserting into an array is costly:

```
flowchart TD
    subgraph InsertMiddle["Inserting at Index 1 in Array [10, 20, 30, 40]"]
        Step1["Original: [ 10, 20, 30, 40, _ ]"]
        Step2["Shift right: [ 10, _, 20, 30, 40 ] (3 items moved!)"]
        Step3["Insert 99: [ 10, 99, 20, 30, 40 ]"]
        Step1 --> Step2 --> Step3
    end
```

Why Insertion and Deletion Take $O(n)$ Time

1. **Inserting at the front (index 0):** You must physically move every existing element one slot to the right to make room. If the array has $1,000,000$ items, that's $1,000,000$ copies ($O(n)$).
2. **Deleting at the front (index 0):** You must physically move every remaining element one slot to the left to close the gap ($O(n)$).
3. **Inserting at the end:** If there is empty capacity, appending at the end is $O(1)$ because no elements need to shift.

Summary of Array Operations

Operation	Index / Location	Time Complexity	Reason
Lookup / Read	Any index `arr[i]`	$O(1)$	Direct address calculation.
Update / Write	Any index `arr[i]`	$O(1)$	Overwrites existing memory slot.
Insert at End	With available space	$O(1)$	No shifting required.
Insert at Front/ Mid	Index 0 or middle	$O(n)$	Must shift downstream elements right.
Delete from Front/Mid	Index 0 or middle	$O(n)$	Must shift downstream elements left.

---

3. Static Array (`int[]`) vs. Dynamic Array (`ArrayList`)

1. Static Array (`int[]`)

- Fixed size at birth. If you declare `new int[5]`, it can never hold 6 elements.
- Extremely memory-efficient; zero pointer overhead.
- Safe and predictable for fixed-size datasets (e.g., standard coordinates or fixed window buffers).

2. Dynamic Array (`ArrayList` in Java, `list` in Python)

- Appears to grow infinitely.
- **How it actually works under the hood:**
  1. Starts with a small internal static array (e.g., capacity 10).
  2. When you append the 11th item, it allocates a **new array with double the capacity** (capacity 20).
  3. It copies all 10 old items into the new array.
  4. It drops the old array for garbage collection.
- **Amortized Analysis ($O(1)$):** Although the doubling step takes $O(n)$ copying work, it happens very infrequently. Across n additions, the total resizing cost is bounded by $2n$. Thus, on average, each append costs only $O(1)$ **amortized time**.

---

4. High-Yield Array Templates for the Midterm

These are standard routines that appear frequently in exam questions. Memorize these simple structures.

Template 1: In-Place Array Reversal (Two-Pointer Technique)
Reverses an array without creating a second array, saving memory:

```
public static void reverseArray(int[] arr) {
    int left = 0;
    int right = arr.length - 1;

    while (left < right) {
        // Swap elements at left and right
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;

        // Move pointers inward
        left++;
        right--;
    }
}
```

- **Time Complexity:** $O(n)$ — visits each element once.
- **Space Complexity:** $O(1)$ — in-place.

Template 2: Finding Extremes (Min and Max)

```
public static int findMin(int[] arr) {
    int minVal = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < minVal) {
            minVal = arr[i];
        }
    }
    return minVal;
}
```

- **Time Complexity:** $O(n)$ — requires exactly $n - 1$ comparisons.

Template 3: Frequency Array / Histogram (Direct Addressing)
Used when counting occurrences of values in a known range (like nucleotide counts `A, C, G, T` or grades `0` to `100`):

```
// Count frequencies of numbers in range 0 to maxVal
public static int[] countFrequencies(int[] arr, int maxVal) {
    int[] count = new int[maxVal + 1]; // All slots initialize to 0
    for (int i = 0; i < arr.length; i++) {
        int val = arr[i];
        count[val]++;
    }
    return count;
}
```

- **Time Complexity:** $O(n)$ — single pass.

- **Space Complexity:** $O(K)$ where K is the maximum value.

Template 4: Prefix Sum Array (Instant Range Queries)
Suppose you have an array and need to answer 10,000 queries asking: *"What is the sum of elements from index L to index R?"* - Naive loop takes $O(n)$ per query $\implies O(Q \cdot n)$ total. - **Prefix Sum Strategy:** Precompute running totals in $O(n)$ once, then answer each query in $O(1)$:

$$\text{Sum}(L, R) = \text{prefix}[R + 1] - \text{prefix}[L]$$

```
public static int[] buildPrefixSum(int[] arr) {
    int n = arr.length;
    int[] prefix = new int[n + 1];
    prefix[0] = 0;
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    return prefix;
}

public static int queryRangeSum(int[] prefix, int L, int R) {
    return prefix[R + 1] - prefix[L];
}
```

- **Build Time:** $O(n)$.
- **Query Time:** $O(1)$ per query.

Chapter 3: Searching Algorithms and Sliding Windows

1. Searching: How to Find What You Need

Searching is finding the position of a specific target value (a **key**) inside a collection, or confirming that the key does not exist.

There are two primary paradigms: 1. **Unsorted Data:** You have no choice but to scan elements one by one (**Linear Search**). 2. **Sorted Data:** You can repeatedly eliminate half of the remaining elements at every step (**Binary Search**).

```
flowchart TD
    subgraph SearchChoice["Searching Strategy Decision"]
        Q["Is the array already sorted?"]
        Q -->|No| LS["Linear Search: O(n)\nMust check each slot one by
one"]
        Q -->|Yes| BS["Binary Search: O(log n)\nHalve search space each
step"]
    end
```

---

2. Linear Search and Sentinel Optimization

1. Standard Linear Search

- **How it works:** Start at index 0. Compare each element to `target`. If matched, return the index. If the end of the array is reached without finding it, return `-1`.
- **Analogy:** Checking test tubes in an unsorted rack one by one from left to right until you find the sample label.

```java
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Found at index i
        }
    }
    return -1; // Not found
}
```

- **Time Complexity:**
  - **Best Case:** $\Theta(1)$ (target is at index 0).
  - **Worst Case:** $\Theta(n)$ (target is at the very last index or not present).
  - **Average Case:** $\Theta(n)$ (checks roughly $n/2$ items).
- **Space Complexity:** $O(1)$.

2. Sentinel Linear Search (Optimized Linear Search)

On every iteration of a standard `for` loop, the processor performs **two checks**: 1. Check loop boundary: `i < arr.length` 2. Check value: `arr[i] == target`

The Sentinel Trick: If we have extra space at the end of the array, we intentionally plant the target value at `arr[n]` (called a **Sentinel**). Now, the target is guaranteed to be found, so we can completely eliminate the `i < n` boundary check from the loop!

```
// Requires an array of capacity n + 1 where slot n is available
public static int sentinelLinearSearch(int[] arr, int n, int target) {
    // 1. Place sentinel at the end
    arr[n] = target;

    // 2. Loop only checks value match (saves 50% of loop comparisons!)
    int i = 0;
    while (arr[i] != target) {
        i++;
    }

    // 3. Check if we found a real match or hit the sentinel
    if (i < n) {
        return i; // Found genuine match in original array
    }
    return -1;    // Only hit the sentinel; target was not originally
present
}
```

- **Asymptotic Complexity:** Still $\Theta(n)$, but runs roughly $1.5\times$ to $2\times$ faster in practice due to half as many branch instructions per iteration.

---

3. Binary Search: The Logarithmic Powerhouse

The Critical Rule

> **Binary Search ONLY works on data that is already SORTED!** If you run binary search on an unsorted array, it will return incorrect results because its fundamental assumption is broken.

How Binary Search Works Step-by-Step

1. Maintain two boundary markers: `low` (start of search region) and `high` (end of search region).
2. Calculate the midpoint: `mid = low + (high - low) / 2`.
3. Compare the value at `arr[mid]` with `target`:
   - If `arr[mid] == target`: Success! Return `mid`.
   - If `arr[mid] < target`: The target must be in the **right half**. Move `low = mid + 1`.
   - If `arr[mid] > target`: The target must be in the **left half**. Move `high = mid - 1`.
4. If `low > high`, the search space has shrunk to zero; the target is not in the array. Return `-1`.

```
flowchart TD
    subgraph BinarySearchFlow["Binary Search Decision at mid"]
        M["Check arr[mid] vs target"]
        M -->|arr[mid] == target| F["Found! Return mid"]
        M -->|arr[mid] < target| R["Target is larger:\nDiscard left half ->
low = mid + 1"]
        M -->|arr[mid] > target| L["Target is smaller:\nDiscard right half
-> high = mid - 1"]
    end
```

Concrete Hand-Trace Example

Search for target `23` in sorted array `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` ($n = 10$).

Iteration	`low`	`high`	`mid`	`arr[mid]`	Comparison	Action Taken
Pass 1	0	9	4	16	$16 < 23$	Target is in right half $\Longrightarrow$ `low = 5`
Pass 2	5	9	7	56	$56 > 23$	Target is in left half $\Longrightarrow$ `high = 6`
Pass 3	5	6	5	23	$23 == 23$	**Match found! Return index 5.**

In only **3 comparisons**, we found the answer in a list of 10 items.

The Integer Overflow Trap: Why Not `(low + high) / 2`?

In Java, 32-bit `int` values have a maximum limit of $2,147,483,647$. If `low` and `high` are both large (e.g., 1.5 billion), adding `low + high` overflows into a negative number, causing an `ArrayIndexOutOfBoundsException`. - **Unsafe:** `int mid = (low + high) / 2;` - **Safe:** `int mid = low + (high - low) / 2;`

Standard Binary Search Template (Iterative)

```
public static int binarySearch(int[] arr, int target) {
    int low = 0;
    int high = arr.length - 1;
```

```
    while (low <= high) { // Notice '<=' not '<'
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid; // Target found
        } else if (arr[mid] < target) {
            low = mid + 1;  // Search right half
        } else {
            high = mid - 1; // Search left half
        }
    }
    return -1; // Target not found
}
```

- **Time Complexity:**
  - **Best Case:** $\Theta(1)$ (middle element is target on first try).
  - **Worst Case:** $\Theta(\log_2 n)$ (halves search space until 1 element remains).
  - **Average Case:** $\Theta(\log_2 n)$.
- **Space Complexity:** $O(1)$ (in-place iterative).

---

4. Exam Variations: First and Last Occurrences

When an array contains **duplicate values** (e.g., `[2, 4, 4, 4, 7, 9]`), standard binary search might return any of the matching indices. Exam questions often ask for the **first (leftmost)** or **last (rightmost)** occurrence.

First Occurrence (Lower Bound)

When `arr[mid] == target`, don't return immediately! Record `mid` as a candidate, and continue searching in the **left half** (`high = mid - 1`):

```
public static int findFirstOccurrence(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            result = mid;       // Record candidate index
            high = mid - 1;     // Keep searching left for earlier
occurrence!
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
```

```
    return result;
}
```

Last Occurrence (Upper Bound)

When `arr[mid] == target`, record `mid` and continue searching in the **right half** (`low = mid + 1`):

```
public static int findLastOccurrence(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            result = mid;       // Record candidate index
            low = mid + 1;      // Keep searching right for later
occurrence!
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}
```

---

5. The Sliding Window Technique

The Biological Concept

In genome analysis, computing GC-content across a chromosome is typically done using a **sliding window of fixed width** k (e.g., 100 bp window moving 1 base at a time).

```
flowchart LR
    subgraph WindowOld["Window at Step 1: [A, C, G] (Sum: 3)"]
        W1["[ A, C, G ] T A"]
    end
    subgraph WindowNew["Window at Step 2: [C, G, T] (Slide by 1)"]
        W2["A [ C, G, T ] A"]
    end
    WindowOld -->|Subtract outgoing 'A', Add incoming 'T'| WindowNew
```

The $O(1)$ Window Update

- **Naive approach:** For each window position, sum all k elements from scratch. Total time = $O(n \cdot k)$.
- **Sliding Window insight:** When the window slides right by 1 position, only **two elements change**:

1. The leftmost element leaves the window (**subtract** it).
2. The new rightmost element enters the window (**add** it).
3. All elements in the middle stay the same!

- Total time drops from $O(n \cdot k)$ to $\boldsymbol{O(n)}$.

Fixed Sliding Window Template: Maximum Sum Subarray of Size k

```java
public static int maxSumWindow(int[] arr, int k) {
    int n = arr.length;
    if (n < k) return -1;

    // 1. Calculate sum of the first window of size k
    int windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    // 2. Slide the window from index k to the end
    for (int i = k; i < n; i++) {
        windowSum += arr[i] - arr[i - k]; // Add incoming, subtract
outgoing
        if (windowSum > maxSum) {
            maxSum = windowSum;
        }
    }
    return maxSum;
}
```

- **Time Complexity:** $O(n)$ — visits each array element at most twice.
- **Space Complexity:** $O(1)$ — only stores running sum variables.

Chapter 4: Elementary Sorting Algorithms (Bubble, Selection, Insertion)

1. Why Sorting Matters & The Concept of Stability

Sorting arranges elements in a specified order (ascending or descending). In biological pipelines and data science, sorting is the crucial step that enables: - Fast $O(\log n)$ binary searching. - Duplicate detection and coordinate-based genome alignment. - Identifying top scoring BLAST hits or Phred quality percentiles.

```
flowchart TD
    subgraph StabilityDemo["The Stability Principle"]
        Orig["Original Array (Order of Arrival):\n[ 5a,  3,  5b,  1 ]"]
        Stable["Stable Sort (5a remains before 5b):\n[ 1,  3,  5a,  5b ]"]
        Unstable["Unstable Sort (5b leapfrogs 5a):\n[ 1,  3,  5b,  5a ]"]
        Orig -->|Preserves tie order| Stable
        Orig -->|Scrambles tie order| Unstable
    end
```

What is Sorting Stability?

An algorithm is **Stable** if elements with identical keys retain their original relative order after sorting. - **Why it matters:** Suppose you sort genomic reads first by sample ID, and then sort by chromosomal start coordinate. A **stable** sort guarantees that reads at the same coordinate remain grouped by sample ID. An **unstable** sort scrambles that grouping. - **Stable Sorts:** Bubble Sort, Insertion Sort, Merge Sort. - **Unstable Sorts:** Selection Sort, Quick Sort, Heap Sort.

---

2. Bubble Sort: Adjacent Pairwise Swaps

How It Works Step-by-Step

1. Walk through the array from left to right.
2. Compare each adjacent pair: `arr[j]` and `arr[j + 1]`.
3. If the left item is strictly greater than the right item (`arr[j] > arr[j + 1]`), **swap them**.
4. After Pass 1, the **largest element** has “bubbled up” to the very end of the array.
5. In Pass 2, repeat the process for all elements except the last one (which is already locked in place).
6. Continue for $n - 1$ passes.

Hand-Trace Example

Input array: `[5, 1, 4, 2, 8]` ($n = 5$)

- **Pass 1:**
  - Compare $(5, 1) \implies 5 > 1$, swap $\implies [1, 5, 4, 2, 8]$
  - Compare $(5, 4) \implies 5 > 4$, swap $\implies [1, 4, 5, 2, 8]$
  - Compare $(5, 2) \implies 5 > 2$, swap $\implies [1, 4, 2, 5, 8]$

- Compare $(5, 8) \implies 5 < 8$, no swap $\implies [1, 4, 2, 5, 8]$
- *End of Pass 1:* `8` *is permanently locked at index 4.*

- **Pass 2:**
  - Compare $(1, 4) \implies$ no swap
  - Compare $(4, 2) \implies$ swap $\implies [1, 2, 4, 5, 8]$
  - Compare $(4, 5) \implies$ no swap
  - *End of Pass 2:* `5` *is permanently locked at index 3.*
- **Pass 3:**
  - Compare $(1, 2) \implies$ no swap
  - Compare $(2, 4) \implies$ no swap
  - *No swaps occurred in this entire pass $\implies$ Array is already sorted! Terminate early.*

The Early-Exit Optimization

By tracking `boolean swapped = false`, if a pass completes without a single swap, the array is verified to be fully sorted. We can break out immediately, achieving $O(n)$ **best-case time**.

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;

        // Inner loop runs up to n - 1 - i because end elements are already
locked
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) { // Strict '>' preserves stability
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }

        // If no two elements were swapped, array is sorted
        if (!swapped) break;
    }
}
```

- **Time Complexity:**
  - **Best Case:** $\Theta(n)$ (already sorted with early-exit).
  - **Worst Case:** $\Theta(n^2)$ (reverse sorted).
  - **Average Case:** $\Theta(n^2)$.
- **Swaps:** Best = 0, Worst = $\frac{n(n-1)}{2}$.
- **Space:** $O(1)$ (in-place).
- **Stability: Stable**.

---

3. Selection Sort: Finding Extremes with Minimum Swaps

How It Works Step-by-Step

1. Divide the array conceptually into a **sorted prefix** (left) and an **unsorted suffix** (right).
2. Scan the entire unsorted region to find the **index of the absolute minimum element**.
3. **Swap** that minimum element with the first element of the unsorted region.
4. Expand the sorted region by one slot and repeat until the array is fully sorted.

```
flowchart LR
    subgraph Pass1["Selection Sort Pass 1: Array [5, 1, 4, 2, 8]"]
        Scan["Scan entire array: Min is '1' at index 1"]
        Swap["Swap arr[0] (5) with arr[1] (1)"]
        Result["Result: [ 1 | 5, 4, 2, 8 ] (1 is locked)"]
        Scan --> Swap --> Result
    end
```

Hand-Trace Example

Input array: `[5, 1, 4, 2, 8]`

- **Pass 1:** Unsorted `[5, 1, 4, 2, 8]`. Minimum is `1` at index 1. Swap with index 0 $\Longrightarrow$ `[1 | 5, 4, 2, 8]`
- **Pass 2:** Unsorted `[5, 4, 2, 8]`. Minimum is `2` at index 3. Swap with index 1 $\Longrightarrow$ `[1, 2 | 4, 5, 8]`
- **Pass 3:** Unsorted `[4, 5, 8]`. Minimum is `4` at index 2. Swap with index 2 (itself) $\Longrightarrow$ `[1, 2, 4 | 5, 8]`
- **Pass 4:** Unsorted `[5, 8]`. Minimum is `5` at index 3. Swap with index 3 $\Longrightarrow$ `[1, 2, 4, 5, 8]`

Exam Variation: Minimum-to-Front vs. Maximum-to-End

- **Standard (Min-to-Front):** Scans for minimum and swaps to `arr[i]`.
- **Exam Variation (Max-to-End):** Scans for maximum and swaps to `arr[n - 1 - i]` (tested in Mock Exam 2).

Standard Selection Sort Template (Min-to-Front)

```java
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;

        // Scan unsorted suffix to find smallest item
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
```

```
        // Swap the found minimum into position i (at most 1 swap per
pass!)
        if (minIdx != i) {
            int temp = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
        }
    }
}
```

Selection Sort Variation (Max-to-End)

```
public static void selectionSortMaxToEnd(int[] arr) {
    int n = arr.length;
    for (int i = n - 1; i > 0; i--) {
        int maxIdx = 0;
        for (int j = 1; j <= i; j++) {
            if (arr[j] > arr[maxIdx]) {
                maxIdx = j;
            }
        }
        // Swap maximum to current end position i
        int temp = arr[i];
        arr[i] = arr[maxIdx];
        arr[maxIdx] = temp;
    }
}
```

Why Selection Sort is Fundamentally UNSTABLE

Consider the array `[ 2a, 2b, 1 ]`: 1. The minimum is `1` at index 2. 2. We swap `arr[0]` (`2a`) with `arr[2]` (`1`). 3. The array becomes `[ 1, 2b, 2a ]`. 4. `2a` has leapfrogged over `2b`! Their original relative order is broken $\implies$ **Unstable**.

- **Time Complexity:** $\Theta(n^2)$ in ALL cases (Best, Worst, Average) because it always scans all remaining elements to verify the minimum.
- **Swaps:** At most $n - 1$ swaps (extremely low write overhead).
- **Space:** $O(1)$.
- **Stability: Unstable**.

---

4. Insertion Sort: The Card Player's Algorithm

How It Works Step-by-Step

1. Imagine holding a hand of playing cards.
2. The first card (`arr[0]`) is considered already sorted.
3. Pick up the next card (`key = arr[i]`).
4. Compare `key` with cards in your sorted hand from right to left.

5. As long as a card in your hand is strictly larger than `key`, **shift it one slot to the right**.
6. When you find a card smaller than or equal to `key`, drop `key` into the open slot.
7. Repeat for all remaining cards.

```
flowchart TD
    subgraph CardSort["Insertion Sort Step: Inserting 2 into [1, 4, 5]"]
        S1["Key = 2. Compare with 5: 5 > 2 -> Shift 5 right: [1, 4, _, 5]"]
        S2["Compare with 4: 4 > 2 -> Shift 4 right: [1, _, 4, 5]"]
        S3["Compare with 1: 1 < 2 -> STOP. Drop 2 into slot: [1, 2, 4, 5]"]
        S1 --> S2 --> S3
    end
```

Hand-Trace Example

Input array: `[5, 1, 4, 2, 8]`

- **Pass 1 ($i = 1$):** `key = 1`. Compare with 5: $5 > 1 \implies$ shift 5 $\implies$ insert 1 at index 0 $\implies$ `[1, 5 | 4, 2, 8]`
- **Pass 2 ($i = 2$):** `key = 4`. Compare with 5: $5 > 4 \implies$ shift 5 $\implies$ compare with 1: $1 < 4 \implies$ insert 4 at index 1 $\implies$ `[1, 4, 5 | 2, 8]`
- **Pass 3 ($i = 3$):** `key = 2`. Compare with 5 (shift), compare with 4 (shift), compare with 1 ($1 < 2$) $\implies$ insert 2 at index 1 $\implies$ `[1, 2, 4, 5 | 8]`
- **Pass 4 ($i = 4$):** `key = 8`. Compare with 5: $5 < 8 \implies$ 0 shifts! $\implies$ `[1, 2, 4, 5, 8]`

Standard Insertion Sort Template (Ascending Order)

```
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Shift elements of arr[0..i-1] that are greater than key to one
position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // Shift right
            j--;
        }
        arr[j + 1] = key; // Place key in vacated slot
    }
}
```

Exam Variation: Insertion Sort in Descending Order (Mock Exam 1)

To sort in descending order (largest first), simply flip the inner condition from `arr[j] > key` to `arr[j] < key`:

```
public static void insertionSortDescending(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
```

```
        int key = arr[i];
        int j = i - 1;

        // Shift elements smaller than key to make room for larger value
        while (j >= 0 && arr[j] < key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

- **Time Complexity:**
  - **Best Case:** $\Theta(n)$ (already sorted; inner loop condition fails immediately on each pass, requiring 0 shifts).
  - **Worst Case:** $\Theta(n^2)$ (reverse sorted; every item shifts all the way to the front).
  - **Average Case:** $\Theta(n^2)$.
- **Space:** $O(1)$.
- **Stability: Stable** (strict inequality `arr[j] > key` never shifts equal elements).

---

5. Master Comparison: Elementary Sorting Algorithms

Metric	Bubble Sort (Early Exit)	Selection Sort	Insertion Sort
Best-Case Time	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$
Average-Case Time	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Worst-Case Time	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Best-Case Comparisons	$n-1$	$\frac{n(n-1)}{2}$	$n-1$
Worst-Case Swaps / Shifts	$\frac{n(n-1)}{2}$ swaps	$\boldsymbol{n-1}$ **swaps**	$\frac{n(n-1)}{2}$ shifts
Auxiliary Space	$O(1)$	$O(1)$	$O(1)$
Stability	**Stable**	**UNSTABLE**	**Stable**
Best Use Case	Academic demonstration	When writing to memory is very expensive	**Nearly-sorted data** or small arrays ($n < 20$)

Chapter 5: Stacks (LIFO), Applications, and Monotonic Patterns

1. The Stack ADT: Last-In, First-Out (LIFO)

A **Stack** is a linear data structure that follows the **Last-In, First-Out (LIFO)** principle: - The last element added is always the first element removed. - **Physical Analogy:** A spring-loaded cafeteria plate dispenser or a vertical stack of laboratory Petri dishes. You can only place a new dish onto the top, and you can only remove the dish currently on top. You cannot pull a dish from the middle or bottom without toppling the stack.

```
flowchart TD
    subgraph StackOperations["Stack Mechanics (LIFO)"]
        Top["[ 30 ]  <-- TOP (Only accessible element)"]
        Mid["[ 20 ]"]
        Bot["[ 10 ]  <-- BOTTOM"]
        Top --- Mid --- Bot
    end
    Push["push(30)"] --> Top
    Top --> Pop["pop() -> returns 30"]
```

The Stack Contract

Method	Behavior	Time Complexity
`push(x)`	Adds item x to the top of the stack	$O(1)$
`pop()`	Removes and returns the top item	$O(1)$
`peek()`	Inspects the top item without removing it	$O(1)$
`isEmpty()`	Returns `true` if stack contains 0 items	$O(1)$
`size()`	Returns current number of items	$O(1)$

Boundary Errors

- **Stack Underflow:** Calling `pop()` or `peek()` when the stack is empty (`isEmpty() == true`).
- **Stack Overflow:** Calling `push()` on a fixed-capacity array stack that is already full.

---

2. Implementations: Array Stack vs. Linked Stack

1. Fixed-Array Stack Implementation

Uses a simple array `data[]` and an integer `top` tracking the index of the uppermost element. - When empty: `top = -1`. - When pushed: `data[++top] = x`. - When popped: `return data[top--]`.

```
public class ArrayStack {
    private int[] data;
```

```
    private int top;
    private int capacity;

    public ArrayStack(int capacity) {
        this.capacity = capacity;
        this.data = new int[capacity];
        this.top = -1; // Empty state
    }

    public void push(int val) {
        if (top == capacity - 1) {
            throw new RuntimeException("Stack Overflow: Stack is full");
        }
        data[++top] = val; // Increment top first, then store
    }

    public int pop() {
        if (isEmpty()) {
            throw new RuntimeException("Stack Underflow: Stack is empty");
        }
        return data[top--]; // Return value, then decrement top
    }

    public int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Stack Underflow: Stack is empty");
        }
        return data[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public int size() {
        return top + 1;
    }
}
```

---

3. High-Yield Exam Applications

Application 1: RNA Secondary Structure & Balanced Parentheses

In molecular biology, an RNA strand folds back onto itself. Stems are formed by complementary base pairs (e.g., `( )` or `[ ]`). A valid RNA stem-loop must never have crossing or unclosed base pairings (pseudoknot-free).

Algorithm: 1. Traverse the string character by character. 2. If you see an opening bracket `(`, `[`, `{`, **push** it onto the stack. 3. If you see a closing bracket `)`, `]`, `}`: - If the stack is empty $\Longrightarrow$ **Unbalanced** (closing bracket without opener). - Pop the

top character from the stack. If it does not match the closing bracket $\Longrightarrow$ **Unbalanced** (mismatched pair). 4. At the end of the string, if the stack is completely empty $\Longrightarrow$ **Balanced**; otherwise **Unbalanced** (unclosed opener remaining).

```java
public static boolean isBalanced(String s) {
    java.util.Stack<Character> stack = new java.util.Stack<>();

    for (int i = 0; i < s.length(); i++) {
        char ch = s.charAt(i);

        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push(ch);
        } else if (ch == ')' || ch == ']' || ch == '}') {
            if (stack.isEmpty()) return false; // Extra closing bracket

            char top = stack.pop();
            if (ch == ')' && top != '(') return false;
            if (ch == ']' && top != '[') return false;
            if (ch == '}' && top != '{') return false;
        }
    }
    return stack.isEmpty(); // Must be empty at end
}
```

- **Time Complexity:** $O(n)$ — single pass through string.
- **Space Complexity:** $O(n)$ — worst-case all openers.

Application 2: Reversing a Sequence Using a Stack

Because a stack is LIFO, pouring data into a stack and popping it out naturally reverses the order:

```java
public static void reverseSequence(int[] arr) {
    java.util.Stack<Integer> stack = new java.util.Stack<>();

    // Push all elements into stack
    for (int num : arr) {
        stack.push(num);
    }

    // Pop back into array; order is now reversed
    for (int i = 0; i < arr.length; i++) {
        arr[i] = stack.pop();
    }
}
```

Application 3: Evaluating Postfix (Reverse Polish) Expressions

In normal arithmetic (infix), operators sit between numbers (`3 + 4`). In postfix notation, operators follow numbers (`3 4 +`). Postfix is unambiguous and requires zero parentheses.

Algorithm: 1. Tokenize expression from left to right. 2. If token is a **number**, push it onto stack. 3. If token is an **operator** (`+` , `-` , `*` , `/`): - Pop operand 2: `b = stack.pop()` . - Pop operand 1: `a = stack.pop()` . - Compute `result = a op b` (notice order: a is left operand, b is right!). - Push `result` back onto stack. 4. Final answer is the last number on the stack.

```
public static int evaluatePostfix(String[] tokens) {
    java.util.Stack<Integer> stack = new java.util.Stack<>();

    for (String token : tokens) {
        if (token.equals("+")) {
            int b = stack.pop();
            int a = stack.pop();
            stack.push(a + b);
        } else if (token.equals("-")) {
            int b = stack.pop();
            int a = stack.pop();
            stack.push(a - b); // a - b, NOT b - a!
        } else if (token.equals("*")) {
            int b = stack.pop();
            int a = stack.pop();
            stack.push(a * b);
        } else if (token.equals("/")) {
            int b = stack.pop();
            int a = stack.pop();
            stack.push(a / b);
        } else {
            stack.push(Integer.parseInt(token)); // Push number
        }
    }
    return stack.pop();
}
```

Application 4: Sorting a Stack Using One Auxiliary Stack (Exam Classic)

> **Problem (from Mock Exam 2):** Given a stack `s1` , sort its elements in ascending order (smallest element at the top) using **only one additional auxiliary stack `s2`** . You may not use any other data structures (no arrays, no lists).

The Mechanism: - Think of `s2` as maintaining a sorted tower with the **largest element at the bottom and smallest element at the top** (or vice versa). - While `s1` is not empty: 1. Pop the top element from `s1` and store in temporary variable

`temp = s1.pop()`. 2. While `s2` is not empty AND `s2.peek() > temp`: - Pop from `s2` and push back into `s1`! (Move obstacles out of the way). 3. Push `temp` onto `s2`. - When `s1` is empty, `s2` is sorted with the largest element on top. Pour `s2` back into `s1`, and `s1` now has the smallest element on top!

```
public static void sortStack(java.util.Stack<Integer> s1) {
    java.util.Stack<Integer> s2 = new java.util.Stack<>();

    while (!s1.isEmpty()) {
        int temp = s1.pop();

        // While temporary element is smaller than s2's top,
        // move elements from s2 back to s1
        while (!s2.isEmpty() && s2.peek() > temp) {
            s1.push(s2.pop());
        }

        s2.push(temp);
    }

    // Transfer back to s1 so smallest is on top
    while (!s2.isEmpty()) {
        s1.push(s2.pop());
    }
}
```

- **Time Complexity:** $O(n^2)$ worst-case (behaves similarly to Insertion Sort).
- **Space Complexity:** $O(n)$ for the single auxiliary stack.

Application 5: Min-Stack ($O(1)$ Minimum Lookup)
Design a stack that supports `push`, `pop`, `peek`, and `getMin()` all in $O(1)$ **time**.

Mechanism: Maintain an auxiliary `minStack`. - On `push(x)`: push x to `mainStack`. Push `Math.min(x, minStack.peek())` to `minStack`. - On `pop()`: pop from both stacks. - On `getMin()`: simply call `minStack.peek()`.

```
public class MinStack {
    private java.util.Stack<Integer> data = new java.util.Stack<>();
    private java.util.Stack<Integer> minStack = new java.util.Stack<>();

    public void push(int val) {
        data.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        } else {
            minStack.push(minStack.peek());
        }
    }
```

```
    public int pop() {
        minStack.pop();
        return data.pop();
    }

    public int getMin() {
        return minStack.peek();
    }
}
```

Chapter 6: Queues (FIFO), Circular Queues, Deques, and Priority Queues

1. The Queue ADT: First-In, First-Out (FIFO)

A **Queue** is a linear data structure that follows the **First-In, First-Out (FIFO)** principle: - The first element added is the first element removed. - **Physical Analogy:** A line of people waiting at a pharmacy counter or a batch of sequencing jobs waiting for alignment on a server. New arrivals join the **rear (tail)**, and the person served first leaves from the **front (head)**.

```
flowchart LR
    Enqueue["enqueue(x)\n(joins at rear)"] --> Rear["[ Rear: 40 ]"]
    Rear --> Mid2["[ 30 ]"]
    Mid2 --> Mid1["[ 20 ]"]
    Mid1 --> Front["[ Front: 10 ]"]
    Front --> Dequeue["dequeue()\n(leaves from front)"]
```

The Queue Contract

Method	Behavior	Time Complexity
`enqueue(x)`	Appends item x to the rear of the queue	$O(1)$
`dequeue()`	Removes and returns the item at the front	$O(1)$ (with proper design)
`peek()`	Inspects front item without removing it	$O(1)$
`isEmpty()`	Returns `true` if queue has 0 elements	$O(1)$
`size()`	Returns current number of elements	$O(1)$

---

2. The Naive Array Queue Flaw: The $O(n)$ Dequeue Penalty

If you implement a queue with a standard array by appending at `arr[size++]`: - `enqueue` is fast ($O(1)$). - But `dequeue` removes `arr[0]`. To keep the remaining elements at the front of the array, you must **shift every remaining element one position to the left** ($O(n)$). - Doing n dequeues results in $\boldsymbol{O(n^2)}$ **total work**, which is unacceptable.

---

3. The Circular Array Queue (Ring Buffer)

To eliminate shifting and achieve $O(1)$ **time for both enqueue and dequeue**, we allow the `front` and `rear` pointers to advance forward through the array. When a pointer hits the end of the array, it wraps back around to index 0 using the **modulo operator (`%`)**.

```
flowchart TD
    subgraph RingBuffer["Circular Array Concept (Capacity = 5)"]
        direction LR
        S0["Index 0\n[ 50 (Rear) ]"] --- S1["Index 1\n[ _ ]"]
        S1 --- S2["Index 2\n[ 20 (Front) ]"]
        S2 --- S3["Index 3\n[ 30 ]"]
        S3 --- S4["Index 4\n[ 40 ]"]
        S4 -.->|wraps around via (rear + 1) % 5| S0
    end
```

The Modulo Wrap-Around Formulas

- Moving rear forward: `rear = (rear + 1) % capacity;`
- Moving front forward: `front = (front + 1) % capacity;`

Disambiguating Full vs. Empty: The `count` Strategy

When `front == rear`, does it mean the queue is completely empty or completely full? The easiest, most bulletproof way to avoid ambiguity on an exam is to maintain an explicit variable: `int count = 0;`. - **Empty:** `count == 0` - **Full:** `count == capacity`

Complete Circular Queue Template

```java
public class CircularQueue {
    private int[] data;
    private int front;
    private int rear;
    private int count;
    private int capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        this.data = new int[capacity];
        this.front = 0;
        this.rear = 0;
        this.count = 0;
    }

    public boolean enqueue(int val) {
        if (isFull()) {
            System.out.println("Queue Overflow: Queue is full");
            return false;
        }
        data[rear] = val;
        rear = (rear + 1) % capacity; // Wrap around to index 0 if at end
        count++;
        return true;
```

```
    }

    public int dequeue() {
        if (isEmpty()) {
            throw new RuntimeException("Queue Underflow: Queue is empty");
        }
        int removed = data[front];
        front = (front + 1) % capacity; // Advance front pointer
        count--;
        return removed;
    }

    public int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Queue is empty");
        }
        return data[front];
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == capacity;
    }

    public int size() {
        return count;
    }
}
```

Hand-Tracing a Circular Queue (Mock Exam 2 Style)

Suppose capacity = 5. Initially `front = 0`, `rear = 0`, `count = 0`. Array: `[ _, _, _, _, _ ]`.

Opera-tion	Value Stored at Index	`front`	`rear`	`count`	Array State
`enqueue(10)`	`data[0] = 10`	0	1	1	`[ 10, _, _, _, _ ]`
`enqueue(20)`	`data[1] = 20`	0	2	2	`[ 10, 20, _, _, _ ]`
`enqueue(30)`	`data[2] = 30`	0	3	3	`[ 10, 20, 30, _, _ ]`
`dequeue()`	Returns `data[0]` (10)	1	3	2	`[ _, 20, 30, _, _ ]`
`enqueue(40)`	`data[3] = 40`	1	4	3	`[ _, 20, 30, 40, _ ]`
`enqueue(50)`	`data[4] = 50`	1	0 (wrapped!)	4	`[ _, 20, 30, 40, 50 ]`
`enqueue(60)`	`data[0] = 60`	1	1	5 (FULL)	`[ 60, 20, 30, 40, 50 ]`

---

4. Linked Queue: Dynamic Memory with Two Pointers

When capacity is unknown, a linked queue avoids fixed-size limitations: - `head` pointer: points to front node (dequeue takes from here in $O(1)$). - `tail` pointer: points to rear node (enqueue attaches here in $O(1)$).

```
public class LinkedQueue {
    private static class Node {
        int val;
        Node next;
        Node(int val) { this.val = val; }
    }

    private Node head = null;
    private Node tail = null;
    private int size = 0;

    public void enqueue(int val) {
        Node newNode = new Node(val);
        if (isEmpty()) {
            head = tail = newNode;
```

```
        } else {
            tail.next = newNode;
            tail = newNode;
        }
        size++;
    }

    public int dequeue() {
        if (isEmpty()) throw new RuntimeException("Empty Queue");
        int val = head.val;
        head = head.next;
        if (head == null) tail = null; // List became empty
        size--;
        return val;
    }

    public boolean isEmpty() { return head == null; }
}
```

---

5. High-Yield Exam Applications

Application 1: Reversing an Entire Queue Using a Stack

Because queues preserve order and stacks invert order, pouring a queue into a stack and back into the queue completely reverses it!

```
public static void reverseQueue(java.util.Queue<Integer> q) {
    java.util.Stack<Integer> s = new java.util.Stack<>();

    // 1. Dequeue all elements into stack
    while (!q.isEmpty()) {
        s.push(q.remove());
    }

    // 2. Pop all elements back into queue
    while (!s.isEmpty()) {
        q.add(s.pop());
    }
}
```

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$

Application 2: Reversing the First k Elements of a Queue

> **Problem:** Given a queue `[10, 20, 30, 40, 50]` and $k = 3$, reverse only the first 3 elements so the queue becomes `[30, 20, 10, 40, 50]`.

Algorithm: 1. Dequeue first k elements and push them onto a stack. 2. Pop all elements from stack and enqueue them back to the queue (they are now at the rear in reversed order). 3. Dequeue the remaining $(n-k)$ elements from the front and immediately re-enqueue them at the rear!

```
public static void reverseFirstK(java.util.Queue<Integer> q, int k) {
    if (q.isEmpty() || k <= 0 || k > q.size()) return;

    java.util.Stack<Integer> stack = new java.util.Stack<>();

    // 1. Push first k elements onto stack
    for (int i = 0; i < k; i++) {
        stack.push(q.remove());
    }

    // 2. Enqueue back from stack to rear
    while (!stack.isEmpty()) {
        q.add(stack.pop());
    }

    // 3. Move remaining (n - k) elements to the rear
    int remaining = q.size() - k;
    for (int i = 0; i < remaining; i++) {
        q.add(q.remove());
    }
}
```

---

6. Priority Queues & Heaps: Priority Over Arrival

Unlike a standard FIFO queue where elements exit by arrival time, a **Priority Queue (PQ)** removes elements based on **priority (key value)**.

```
flowchart TD
    subgraph PQConcept["Priority Queue Extraction"]
        MaxPQ["Max Priority Queue\nExtracts MAXIMUM value first\n(e.g.,
highest Phred quality score)"]
        MinPQ["Min Priority Queue\nExtracts MINIMUM value first\n(e.g.,
shortest physical distance)"]
    end
```

Core Operations

- `insert(x)` : Adds item x ($\Theta(\log n)$ using binary heap).
- `extractMax()` or `extractMin()` : Removes and returns the highest priority item ($\Theta(\log n)$).
- `peek()` : Inspects highest priority item without removing it ($\Theta(1)$).

Exam Tracing Rules

On midterm exams (such as Mock Exam 1 & Mock Exam 2), you will be asked to trace values exiting a PQ: - In a **Max Priority Queue**, whenever you call `extractMax()`, scan all active items and remove the largest number. - In a **Min Priority Queue**, whenever you call `extractMin()`, scan all active items and remove the smallest number.

Chapter 7: Linked Lists in Depth: Nodes, Pointers, and Invariants

1. Why Linked Lists? The Non-Contiguous Memory Model

In an array, all items must sit in contiguous (strictly adjacent) physical memory slots. While this guarantees instantaneous $O(1)$ indexing, it has severe drawbacks: 1. Resizing requires allocating a brand-new array and copying all elements ($O(n)$). 2. Inserting or deleting at the front requires shifting every element downstream ($O(n)$). 3. If computer memory is fragmented into small chunks, allocating a 2 GB contiguous array may fail even if total free RAM exceeds 8 GB.

A **Linked List** completely abandons contiguous memory. - Items are stored in independent memory containers called **Nodes**. - Each node lives wherever free space exists in the Heap. - Nodes are linked together into a sequence by storing the memory address of the next node (**the `next` pointer/reference**).

```
flowchart LR
    subgraph ArrayLayout["Array: Locked Contiguously in RAM"]
        direction LR
        A0["[ 10 ]"] --- A1["[ 20 ]"] --- A2["[ 30 ]"]
    end

    subgraph LinkedListLayout["Linked List: Scattered Anywhere in Heap"]
        direction LR
        N1["Node at 0x100\n[ 10 | next ]"] -->|Points to 0x850| N2["Node at
0x850\n[ 20 | next ]"]
        N2 -->|Points to 0x420| N3["Node at 0x420\n[ 30 | null ]"]
    end
```

The Train Car Metaphor

Think of freight train cars. - Each car can carry cargo (`val`). - Cars do not need to be manufactured in a single giant solid block of steel. - Each car simply has a coupling hook (`next`) connected to the front of the next car. - To insert a new car into the middle of a train, you don’t need to rebuild or move the entire train; you simply uncouple one hook and attach two new hooks ($O(1)$ pointer splice).

---

2. Array vs. Linked List: The Fundamental Trade-Offs

Property	Array (`int[]`)	Singly Linked List
Physical Memory	Contiguous block in RAM	Scattered Heap nodes connected by references
Random Access (`get(i)`)	$O(1)$ (instant formula)	$O(n)$ (must walk from head node-by-node)
Insert / Delete at Front	$O(n)$ (must shift all items)	$O(1)$ (update head pointer)
Insert / Delete at Tail	$O(1)$ (if capacity exists)	$O(1)$ (with tail pointer), $O(n)$ (without)
Insert / Delete in Middle	$O(n)$ (shifting)	$O(1)$ (once pointer to node is in hand)
Memory Overhead	Minimal (pure data)	High (extra 8-byte pointer per node)
Binary Search Support	**Yes** ($O(\log n)$)	**NO!** (cannot jump to middle in $O(1)$)

---

3. Singly Linked List Mechanics

The Node Class

```
public class Node {
    public int val;     // The data payload
    public Node next;   // Reference to the next Node (null if last node)

    public Node(int val) {
        this.val = val;
        this.next = null;
    }

    public Node(int val, Node next) {
        this.val = val;
        this.next = next;
    }
}
```

Traversal Pattern (The Standard Loop)

To visit every node in a linked list, start at `head` and advance until `curr == null`:

```
public static void printList(Node head) {
    Node curr = head;
    while (curr != null) {
        System.out.print(curr.val + " -> ");
```

```
        curr = curr.next; // Advance to next node
    }
    System.out.println("null");
}
```

Inserting at the Head ($O(1)$)

1. Create a new node.
2. Point `newNode.next` to the current `head`.
3. Update `head` to point to `newNode`.

```
public static Node insertAtHead(Node head, int val) {
    Node newNode = new Node(val);
    newNode.next = head;
    head = newNode;
    return head;
}
```

Deleting a Node by Value

To delete a node with target value: - If target is at `head`: advance `head = head.next`. - Otherwise, traverse using a pointer `curr`. Stop when `curr.next.val == target`. - Splice around the node: `curr.next = curr.next.next;`.

```
flowchart LR
    A["Node A\n(curr)"] -->|Old link| B["Node B\n(to delete)"]
    B --> C["Node C\n(curr.next.next)"]
    A -.->|New spliced link: curr.next = curr.next.next| C
```

---

4. The Sentinel (Dummy Head) Node: Eliminating Edge Cases

One of the biggest causes of bugs on coding exams is forgetting edge cases: - Inserting into an empty list (`head == null`). - Deleting the very first node (`head`). - Updating the list when only 1 node exists.

The Sentinel Technique: Create a dummy node with dummy data (`new Node(-1)`) whose `next` points to the real `head`. - Now, the list is **never empty**, and the real head is simply `dummy.next`! - Every node in the list (even the first real node) now has a valid node before it. - After operations finish, simply return `dummy.next`.

```
public static Node deleteValueWithSentinel(Node head, int target) {
    Node dummy = new Node(-1);
    dummy.next = head;
    Node curr = dummy;

    while (curr.next != null) {
        if (curr.next.val == target) {
            curr.next = curr.next.next; // Delete node
```

```
            break; // Stop after first match
        } else {
            curr = curr.next;
        }
    }
    return dummy.next; // Return true head
}
```

---

5. Doubly Linked Lists: Two-Way Navigation

A **Doubly Linked List** adds a `prev` reference to every node: - `next`: Points to subsequent node. - `prev`: Points to preceding node.

```
flowchart LR
    N1["[ prev | 10 | next ]"] <--> N2["[ prev | 20 | next ]"] <-->
N3["[ prev | 30 | next ]"]
```

The Superpower of Doubly Linked Lists

If you are handed a direct reference to a `Node target`, you can delete it in $O(1)$ **time** without traversing from the head, because the node itself knows its predecessor:

```
public static void deleteNode(Node target) {
    if (target.prev != null) target.prev.next = target.next;
    if (target.next != null) target.next.prev = target.prev;
}
```

---

6. High-Yield Exam Templates

Template 1: Reversing a Singly Linked List (3-Pointer Dance)

> **Problem:** Reverse a linked list in-place in $O(n)$ time and $O(1)$ memory.

Mental Model: Maintain 3 pointers: 1. `prev` (tracks node behind, starts as `null`). 2. `curr` (current node being flipped, starts as `head`). 3. `next` (saves remaining list before we sever the link).

```
public static Node reverseList(Node head) {
    Node prev = null;
    Node curr = head;

    while (curr != null) {
        Node nextTemp = curr.next; // 1. Save next node
        curr.next = prev;          // 2. Reverse arrow!
        prev = curr;               // 3. Move prev forward
```

```
            curr = nextTemp;            // 4. Move curr forward
        }
        return prev; // prev is now the new head!
    }
```

- **Time Complexity:** $O(n)$.
- **Space Complexity:** $O(1)$.

Template 2: Removing Duplicates from a Sorted Linked List (Mock Exam 2)

> **Problem:** Given a sorted linked list `1 -> 1 -> 2 -> 3 -> 3 -> null`, remove all duplicate values so each element appears only once.

Algorithm: - While `curr != null && curr.next != null`: - If `curr.val == curr.next.val`: splice out `curr.next` (`curr.next = curr.next.next`). **Do NOT advance `curr`** (the newly connected node might also be a duplicate!). - Else: advance `curr = curr.next`.

```
public static Node removeDuplicates(Node head) {
    Node curr = head;
    while (curr != null && curr.next != null) {
        if (curr.val == curr.next.val) {
            curr.next = curr.next.next; // Skip duplicate node
        } else {
            curr = curr.next;           // Advance only when values differ
        }
    }
    return head;
}
```

- **Time Complexity:** $O(n)$.
- **Space Complexity:** $O(1)$.

Template 3: Merging Two Sorted Linked Lists (Real Exam 2023)

> **Problem:** Given two sorted lists `L1` and `L2`, merge them into a single sorted list without creating new nodes.

Algorithm: - Use a `dummy` sentinel node. - Compare `l1.val` and `l2.val`. Attach smaller node to `curr.next` and advance that list. - When one list is exhausted, attach the remainder of the other list directly!

```java
public static Node mergeTwoLists(Node l1, Node l2) {
    Node dummy = new Node(-1);
    Node curr = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            curr.next = l1;
            l1 = l1.next;
        } else {
            curr.next = l2;
            l2 = l2.next;
        }
        curr = curr.next;
    }

    // Attach remaining elements
    if (l1 != null) curr.next = l1;
    if (l2 != null) curr.next = l2;

    return dummy.next;
}
```

- **Time Complexity:** $O(n + m)$ where n, m are the lengths of the two lists.
- **Space Complexity:** $O(1)$ auxiliary (reuses existing nodes).

Template 4: Detecting Cycles (Floyd's Tortoise and Hare)

Problem: Determine if a linked list contains a loop (cycle).

Algorithm: - Send two pointers: `slow` moves 1 step at a time, `fast` moves 2 steps at a time. - If `fast` or `fast.next` hits `null`, there is no cycle. - If there is a cycle, `fast` will eventually lap `slow` and they will point to the exact same node (`slow == fast`).

```java
public static boolean hasCycle(Node head) {
    if (head == null || head.next == null) return false;

    Node slow = head;
    Node fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;          // 1 step
        fast = fast.next.next;     // 2 steps

        if (slow == fast) {
            return true; // Cycle detected!
        }
    }
```

```
    return false; // Reached end of list; no cycle
}
```

- **Time Complexity:** $O(n)$.
- **Space Complexity:** $O(1)$.

Chapter 8: Recursion & The Call-Stack Mechanics

1. The Recursive Mental Model: Self-Similarity and Contracts

Recursion is a technique where a method solves a larger problem by calling itself to solve smaller instances of the exact same problem.

```
flowchart TD
    subgraph RussianDolls["Russian Nesting Dolls (Recursion Metaphor)"]
        D1["Big Doll: solve(n)\nCalls solve(n-1)"] --> D2["Medium Doll:
solve(n-1)\nCalls solve(n-2)"]
        D2 --> D3["Small Doll: solve(1)\nCalls solve(0)"]
        D3 --> D0["Tiny Solid Baby Doll: solve(0)\nBASE CASE: Returns
direct answer!"]
    end
```

The Matryoshka (Russian Doll) Metaphor

Imagine opening a wooden nesting doll. Inside, you find an identical doll, only slightly smaller. You keep opening dolls until you reach the smallest, solid baby doll. That baby doll cannot be opened further (the **Base Case**). You then put the wooden shells back together in reverse order, returning to the outside.

The Contract Mindset

When writing or reading a recursive method on an exam: 1. **Define the Contract:** Clearly state what `solve(input)` promises to return. 2. **Trust the Subproblem:** Assume `solve(smallerInput)` **already works correctly**. 3. **Combine:** Ask yourself: *"If someone gives me the solution to the smaller input, how do I compute the solution for the current input?"*

---

2. The Three Mandatory Rules of Recursion

Every valid recursive algorithm must fulfill three rules:

```
flowchart TD
    subgraph Pillars["The 3 Pillars of Recursion"]
        P1["1. Base Case\nThe stopping condition solved directly without
recursion."]
        P2["2. Recursive Step\nCalling the function itself on a strictly
smaller subproblem."]
        P3["3. Definite Progress\nGuaranteeing that every call gets closer
to the Base Case."]
        P1 --> P2 --> P3
    end
```

- If you omit the base case, or if the recursive call does not reduce the problem size (e.g., calling `f(n)` instead of `f(n - 1)`), the program loops forever until the JVM runs out of call stack memory and crashes with a **`StackOverflowError`**.

---

3. Under the Hood: The JVM Call Stack

How does the computer keep track of all these nested calls? Every time a Java method is called, the JVM creates a **Stack Frame** (activation record) on top of the **Call Stack**.

What Lives Inside a Stack Frame?

1. The method's parameters (e.g., `n = 3`).
2. Local variables.
3. The return address (where to jump back to when this method completes).

```
flowchart TD
    subgraph Phase1["Phase 1: Descent (Pushing Frames)"]
        F3["factorial(1) -> Hits Base Case: Returns 1"]
        F2["factorial(2) -> Waits for factorial(1)"]
        F1["factorial(3) -> Waits for factorial(2)"]
        F1 --- F2 --- F3
    end

    subgraph Phase2["Phase 2: Unwinding (Popping Frames)"]
        U3["factorial(1) pops, returns 1 to frame 2"]
        U2["factorial(2) computes 2 * 1 = 2, pops, returns 2 to frame 1"]
        U1["factorial(3) computes 3 * 2 = 6, pops, returns 6 to caller!"]
        U3 --> U2 --> U1
    end
```

Execution Has Two Distinct Phases:

1. **Descent (Diving Down):** Pushing frames onto the Call Stack until the base case is reached.
2. **Unwinding (Bubbling Up):** Popping frames off the Call Stack in reverse order, calculating return values and returning them to the caller.

---

4. Classic Recursive Examples & Walkthroughs

Example 1: Factorial ($n!$)

Mathematical definition: - Base case: $0! = 1$ - Recursive step: $n! = n \times (n-1)!$

```java
public static int factorial(int n) {
    // 1. Base Case
    if (n <= 1) {
        return 1;
    }
    // 2. Recursive Step with definite progress (n - 1)
    return n * factorial(n - 1);
}
```

- **Time Complexity:** $O(n)$ — makes n total calls.
- **Space Complexity:** $O(n)$ — creates n stack frames simultaneously.

Example 2: Fibonacci Numbers — The Exponential Danger

Mathematical definition: - $F(0) = 0, \quad F(1) = 1$ - $F(n) = F(n-1) + F(n-2)$

```java
public static int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2); // 2 recursive branches!
}
```

```mermaid
flowchart TD
    subgraph FibTree["Redundant Computation in Naive Fibonacci: fib(4)"]
        F4["fib(4)"] --> F3["fib(3)"]
        F4 --> F2A["fib(2)"]
        F3 --> F2B["fib(2)"]
        F3 --> F1A["fib(1)"]
        F2A --> F1B["fib(1)"]
        F2A --> F0A["fib(0)"]
        F2B --> F1C["fib(1)"]
        F2B --> F0B["fib(0)"]
    end
```

Why Naive Fibonacci is $O(2^n)$

Every single call branches into two sub-calls. Notice how `fib(2)` is computed multiple times independently! - **Call Tree Shape:** Binary tree of depth n. - **Total Calls:** Approximately 2^n calls $\implies$ **Exponential $O(2^n)$ time**. - For $n = 50$, naive recursion takes days. An iterative loop or dynamic programming array solves it in $O(n)$ time!

Example 3: Binary Search (Recursive Implementation)

```java
public static int binarySearchRecursive(int[] arr, int target, int low, int
high) {
    // Base Case 1: Search region exhausted
    if (low > high) {
        return -1;
    }

    int mid = low + (high - low) / 2;

    // Base Case 2: Target found
    if (arr[mid] == target) {
        return mid;
    }

    // Recursive Step: Search appropriate half
    if (arr[mid] < target) {
        return binarySearchRecursive(arr, target, mid + 1, high);
    } else {
        return binarySearchRecursive(arr, target, low, mid - 1);
```

```
    }
}
```

- **Time Complexity:** $O(\log_2 n)$.
- **Space Complexity:** $O(\log_2 n)$ auxiliary space on the Call Stack.

---

5. Towers of Hanoi: The Masterclass in Recursion

The Problem

You have 3 pegs: **Source (S)**, **Destination (D)**, and **Auxiliary (A)**. There are n disks of varying sizes stacked on S, sorted by size with the smallest disk on top. - **Rules:** 1. Only one disk can be moved at a time. 2. A larger disk may never be placed on top of a smaller disk. 3. Move all n disks from S to D using A.

```
flowchart LR
    subgraph HanoiStrategy["3-Step Recursive Strategy for n Disks"]
        S1["Step 1:\nRecursively move (n - 1) disks\nfrom Source to
Auxiliary (using Dest)"]
        S2["Step 2:\nMove disk n (largest)\ndirectly from Source to Dest"]
        S3["Step 3:\nRecursively move (n - 1) disks\nfrom Auxiliary to Dest
(using Source)"]
        S1 --> S2 --> S3
    end
```

The Java Code

```
public class Hanoi {
    public static void solveHanoi(int n, char source, char dest, char aux)
{
        // Base Case: 1 disk can be moved directly
        if (n == 1) {
            System.out.println("Move disk 1 from " + source + " to " +
dest);
            return;
        }

        // Step 1: Move top n - 1 disks from source to aux peg
        solveHanoi(n - 1, source, aux, dest);

        // Step 2: Move the nth (largest) disk directly to dest
        System.out.println("Move disk " + n + " from " + source + " to " +
dest);

        // Step 3: Move the n - 1 disks from aux to dest peg
        solveHanoi(n - 1, aux, dest, source);
    }
}
```

Tracing $n = 3$ Disks (Total $2^3 - 1 = 7$ Moves)

1. Move disk 1 from S to D
2. Move disk 2 from S to A
3. Move disk 1 from D to A
4. Move disk 3 from S to D *(Largest disk reaches final destination!)*
5. Move disk 1 from A to S
6. Move disk 2 from A to D
7. Move disk 1 from S to D

Complexity Analysis

- **Recurrence Relation:** $T(n) = 2T(n-1) + 1$
- **Total Moves:** $2^n - 1$ moves.
- **Time Complexity:** $\Theta(2^n)$ (Exponential).
- **Space Complexity:** $\Theta(n)$ (maximum depth of Call Stack).

Chapter 9: Merge Sort: The Divide-and-Conquer Masterclass

1. The Divide-and-Conquer Paradigm

Elementary sorting algorithms (Bubble, Selection, Insertion) have quadratic $\Theta(n^2)$ worst-case running times. If you have $10,000,000$ sequencing reads, an $O(n^2)$ algorithm would take hours or days.

To break past the quadratic barrier, computer science uses the **Divide-and-Conquer** strategy:

```
flowchart TD
    subgraph DAndC["Divide-and-Conquer Framework"]
        S1["1. DIVIDE\nCut the problem in half at the midpoint: mid = low +
(high - low)/2"]
        S2["2. CONQUER\nRecursively sort both halves.\nBase case: Subarrays
of size 0 or 1 are ALREADY sorted!"]
        S3["3. COMBINE (MERGE)\nInterleave the two sorted halves into a
single fully sorted array."]
        S1 --> S2 --> S3
    end
```

In Merge Sort: - The **Divide** step is effortless (just calculating `mid`). - The **Conquer** step is recursive. - The **Combine (Merge)** step does all the actual heavy lifting!

---

2. The Core Merging Engine: Interleaving Two Sorted Halves in $O(n)$

The Two Card Piles Metaphor

Imagine you are holding two piles of lab assay cards. Both piles are **already sorted** in ascending order: - Pile L: `[2, 5, 8]` - Pile R: `[1, 6, 9]`

How do you merge them into a single sorted pile? 1. Look at the top card of both piles: compare 2 and 1. 2. Take the smaller card (1 from Pile R), place it face down in the output pile, and expose the next card in Pile R (6). 3. Compare 2 and 6. Take 2, place it in output. 4. Repeat until one pile is empty, then pour all remaining cards from the other pile directly into the output!

```
flowchart TD
    subgraph MergePointers["Merging Two Sorted Arrays"]
        L["Left Subarray:  [ 2,  5,  8 ] (pointer i)"]
        R["Right Subarray: [ 1,  6,  9 ] (pointer j)"]
        Out["Output Array: [ 1,  2,  5,  6,  8,  9 ]"]
        L -.->|Pick smaller| Out
        R -.->|Pick smaller| Out
    end
```

Because every comparison places at least one element into its final position, merging two subarrays of total size n takes strictly $O(n)$ **linear time**.

---

3. Merge Implementations: Standard vs. Sentinel Method

Method 1: Standard Two-Pointer Merge (Recommended for Exams)

This is the most intuitive and robust implementation to write by hand on an exam:

```
public static void merge(int[] arr, int low, int mid, int high) {
    // 1. Calculate lengths of left and right subarrays
    int n1 = mid - low + 1;
    int n2 = high - mid;

    // 2. Allocate temporary scratchpad arrays
    int[] left = new int[n1];
    int[] right = new int[n2];

    // 3. Copy data to temporary arrays
    for (int i = 0; i < n1; i++) left[i] = arr[low + i];
    for (int j = 0; j < n2; j++) right[j] = arr[mid + 1 + j];

    // 4. Interleave elements back into arr[low..high]
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) {
        if (left[i] <= right[j]) { // '<=' ensures stability!
            arr[k] = left[i];
            i++;
        } else {
            arr[k] = right[j];
            j++;
        }
        k++;
    }

    // 5. Copy any remaining elements of left[]
    while (i < n1) {
        arr[k++] = left[i++];
    }

    // 6. Copy any remaining elements of right[]
    while (j < n2) {
        arr[k++] = right[j++];
    }
}
```

Method 2: The Sentinel Method (CLRS & Class Exercise)

In the CLRS textbook (taught in Session 5), to avoid writing the leftover cleanup `while` loops, a **Sentinel value** (∞, represented by `Integer.MAX_VALUE`) is placed at the end of both temporary arrays.

Because ∞ is larger than any real number, when one array runs out of real elements, the comparison `left[i] <= right[j]` will naturally pick all remaining elements from the other array:

```java
public static void mergeSentinel(int[] arr, int low, int mid, int high) {
    int n1 = mid - low + 1;
    int n2 = high - mid;

    // Allocate arrays with +1 slot for sentinel
    int[] left = new int[n1 + 1];
    int[] right = new int[n2 + 1];

    for (int i = 0; i < n1; i++) left[i] = arr[low + i];
    for (int j = 0; j < n2; j++) right[j] = arr[mid + 1 + j];

    // Place sentinels at the ends
    left[n1] = Integer.MAX_VALUE;
    right[n2] = Integer.MAX_VALUE;

    int i = 0, j = 0;
    for (int k = low; k <= high; k++) {
        if (left[i] <= right[j]) {
            arr[k] = left[i++];
        } else {
            arr[k] = right[j++];
        }
    }
}
```

---

4. The Complete Merge Sort Method

The recursive sorting method itself is remarkably short and elegant:

```java
public static void mergeSort(int[] arr, int low, int high) {
    // Base Case: If subarray has 0 or 1 element, it is already sorted!
    if (low >= high) {
        return;
    }

    // 1. Divide: Find midpoint (preventing overflow)
    int mid = low + (high - low) / 2;

    // 2. Conquer: Recursively sort left and right halves
    mergeSort(arr, low, mid);
    mergeSort(arr, mid + 1, high);

    // 3. Combine: Merge the two sorted halves together
    merge(arr, low, mid, high);
}
```

5. Visualizing the Recursion Tree

Trace on array `[38, 27, 43, 3, 9, 82, 10]` ($n = 7$):

```
flowchart TD
    subgraph TreeSplit["Divide Phase (Down) & Merge Phase (Up)"]
        Top["[ 38, 27, 43, 3, 9, 82, 10 ]"]
        L1["[ 38, 27, 43, 3 ]"]
        R1["[ 9, 82, 10 ]"]
        Top --> L1
        Top --> R1

        L2A["[ 38, 27 ]"]
        L2B["[ 43, 3 ]"]
        L1 --> L2A
        L1 --> L2B

        R2A["[ 9, 82 ]"]
        R2B["[ 10 ]"]
        R1 --> R2A
        R1 --> R2B

        L3A["[ 38 ]"]
        L3B["[ 27 ]"]
        L2A --> L3A
        L2A --> L3B

        L3C["[ 43 ]"]
        L3D["[ 3 ]"]
        L2B --> L3C
        L2B --> L3D

        R3A["[ 9 ]"]
        R3B["[ 82 ]"]
        R2A --> R3A
        R2A --> R3B
    end
```

Unwinding and Merging Back Up:

- `[38]` and `[27]` merge $\Longrightarrow [27, 38]$
- `[43]` and `[3]` merge $\Longrightarrow [3, 43]$
- `[27, 38]` and `[3, 43]` merge $\Longrightarrow [3, 27, 38, 43]$
- `[9]` and `[82]` merge $\Longrightarrow [9, 82]$
- `[9, 82]` and `[10]` merge $\Longrightarrow [9, 10, 82]$
- Final merge: `[3, 27, 38, 43]` and `[9, 10, 82]` $\Longrightarrow [3, 9, 10, 27, 38, 43, 82]$

6. Mathematical Analysis & Invariants

1. Why Time Complexity is $\Theta(n \log_2 n)$ in ALL Cases

- **Tree Depth:** Each step cuts array size in half: $n \to n/2 \to n/4 \to ... \to 1$. The depth of the recursion tree is strictly $\mathbf{\log_2 n}$ **levels**.
- **Work Per Level:** At any single level of the tree, merging all the subarrays at that level touches every element in the array once $\implies \mathbf{O(n)}$ work per level.
- **Total Work:** $(\text{Levels}) \times (\text{Work per level}) = \log_2 n \times n = \mathbf{\Theta(n \log_2 n)}$.
- **No Best or Worst Case Variation:** Unlike Quick Sort or Bubble Sort, Merge Sort divides blindly in half and merges every single time. Its best, average, and worst cases are all strictly $\mathbf{\Theta(n \log n)}$.

2. Space Complexity: Why Merge Sort Needs $\Theta(n)$ Auxiliary Space

Unlike Bubble Sort, Selection Sort, or Insertion Sort which run in-place ($O(1)$ space), Merge Sort requires temporary arrays (`left[]` and `right[]`) to hold values during merging. - Total extra memory allocated at the widest merge step is equal to n. - Thus, **Auxiliary Space Complexity is $\Theta(n)$**.

3. Stability Proof

Merge Sort is **Stable** as long as you use `left[i] <= right[j]` in the merge loop. - When two values are equal, the tie is broken by taking the element from `left[]` first. - Since elements in `left[]` originally appeared before elements in `right[]`, their original relative order is preserved.

Chapter 10: Midterm Synthesis, Master Tables, and Decision Trees

1. Master Complexity Reference Tables

Table 1: Sorting Algorithms Side-by-Side

Algorithm	Best Time	Average Time	Worst Time	Comparisons (Worst)	Swaps / Shifts (Worst)	Auxiliary Space	Stable?	When to Use
Bubble Sort (Early Exit)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$ swaps	$O(1)$	**Yes**	Academic tracing; nearly sorted data.
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\frac{n(n-1)}{2}$	$\boldsymbol{n-1}$ **swaps**	$O(1)$	**NO**	When writing to memory is extremely slow/ expensive.
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$ shifts	$O(1)$	**Yes**	Small lists ($n < 20$) or **nearly-sorted datasets**.
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\approx n \log_2 n$	Merging buffer	$\boldsymbol{\Theta(n)}$	**Yes**	Large datasets where guaranteed $O(n \log n)$ is needed.

Table 2: Searching Algorithms Side-by-Side

Algorithm	Best Time	Worst Time	Space	Preconditions / Requirements
Linear Search	$\Theta(1)$	$\Theta(n)$	$O(1)$	Works on completely unsorted data.
Sentinel Linear Search	$\Theta(1)$	$\Theta(n)$	$O(1)$	Saves boundary comparisons by planting target at index n.
Binary Search (Iterative)	$\Theta(1)$	$\Theta(\log n)$	$O(1)$	**Must be SORTED and support $O(1)$ indexing (Arrays).**
Binary Search (Recursive)	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	Consumes call stack frames.
First / Last Occurrence	$\Theta(\log n)$	$\Theta(\log n)$	$O(1)$	Handles duplicate elements in sorted arrays.

Table 3: Linear Data Structures Side-by-Side

Data Structure	Access `get(i)`	Insert at Front	Insert at End	Delete at Front	Delete at End	Search Value	Space Overhead
Static Array (`int[]`)	$O(1)$	$O(n)$	$O(1)$ (if open)	$O(n)$	$O(1)$	$O(n)$	Zero pointer overhead
Dynamic Array (`ArrayList`)	$O(1)$	$O(n)$	**$O(1)$ amortized**	$O(n)$	$O(1)$	$O(n)$	Occasional resize copying
Singly Linked List (no tail)	$O(n)$	**$O(1)$**	$O(n)$	**$O(1)$**	$O(n)$	$O(n)$	1 pointer (`next`) per node
Singly Linked List (with tail)	$O(n)$	**$O(1)$**	**$O(1)$**	**$O(1)$**	$O(n)$	$O(n)$	1 pointer per node + tail
Doubly Linked List	$O(n)$	**$O(1)$**	**$O(1)$**	**$O(1)$**	**$O(1)$**	$O(n)$	2 pointers (`prev`, `next`)
Array Stack (LIFO)	Top: $O(1)$	Push: $O(1)$	—	Pop: $O(1)$	—	$O(n)$	Fixed capacity
Circular Queue (FIFO)	Front: $O(1)$	Enqueue: $O(1)$	—	Dequeue: $O(1)$	—	$O(n)$	Modulo arithmetic

---

2. The 10 Classic Midterm Traps Exposed

These 10 mistakes account for the vast majority of lost marks on DSA exams. Memorize them to avoid falling into them:

```
flowchart TD
    subgraph Traps["Top 4 Deadly Exam Pitfalls"]
```

```
        T1["Trap 1: Binary Search on Unsorted Data\nNever use binary search
unless array is sorted!"]
        T2["Trap 2: Severing Pointers in Linked List\nAlways save nextTemp
= curr.next before rewiring!"]
        T3["Trap 3: Selection Sort is Unstable\nLong-distance swaps
leapfrog equal keys!"]
        T4["Trap 4: Naive Array Queue is O(n) Dequeue\nMust use Circular
Queue with % modulo arithmetic!"]
    end
```

1. **The Binary Search Unsorted Trap:**
   - Attempting binary search on unsorted data gives wrong answers. If data is unsorted, either use linear search ($O(n)$) or sort first ($O(n \log n)$).
2. **The Integer Overflow Mid Trap:**
   - Writing `int mid = (low + high) / 2;` overflows when `low + high > 2,147,483,647`.
   - **Fix:** Always write `int mid = low + (high - low) / 2;`.
3. **The Selection Sort Instability Trap:**
   - Selection sort is **unstable** because swapping the global minimum with `arr[i]` can cause equal elements to leapfrog over each other (e.g., `[2a, 2b, 1]` becomes `[1, 2b, 2a]`).
4. **The Naive Array Queue Dequeue Penalty:**
   - Removing from index 0 of an array requires shifting $n-1$ items left ($O(n)$). A true queue must be either a **Circular Array (Ring Buffer)** using `% capacity` or a **Linked Queue**.
5. **The Lost Pointer in Linked List Insertion:**
   - When inserting node `N` between `A` and `B`:
     - Wrong: `A.next = N; N.next = A.next;` (You just lost `B` forever!).
     - Correct: First connect `N.next = A.next;` then connect `A.next = N;`.
6. **The Premature Advancement Trap in Deduplication:**
   - When deleting duplicates in a sorted linked list: if you delete `curr.next`, **do not advance `curr = curr.next` in that same iteration**! The newly connected neighbor may also be a duplicate.
7. **The Missing Base Case in Recursion:**
   - Forgetting `if (n <= 1) return ...;` causes infinite recursion and throws `StackOverflowError`.
8. **The Object Equality Trap in Java (`==` vs. `.equals()`):**
   - In Java, `==` compares **memory addresses**, not contents!
   - `s1 == s2` checks if they are the exact same object in the Heap.
   - `s1.equals(s2)` checks if the text contents match.
9. **The Postfix Operator Operand Inversion:**
   - In postfix evaluation:
     - First popped item is the **right operand** (`b = stack.pop()`).
     - Second popped item is the **left operand** (`a = stack.pop()`).
     - For subtraction and division, compute `a - b` and `a / b` (NOT `b - a` or `b / a`!).

10. **The Circular Queue Capacity Disambiguation:**
    - When `front == rear`, the queue could be completely empty OR completely full.
    - **Fix:** Maintain an explicit `int count = 0;` variable to know immediately.

---

3. Algorithmic Decision Tree: What Should You Use?

Use this decision tree when answering exam design questions:

```
flowchart TD
    Start["What is your primary requirement?"]

    Start --> Q1["Fast Lookups by Index (0..n-1)"]
    Q1 --> A1["Use an Array / ArrayList\nO(1) random access"]

    Start --> Q2["Frequent Insertions / Deletions at Ends or Middle"]
    Q2 --> A2["Use a Linked List\nO(1) pointer splices, no shifting"]

    Start --> Q3["Processing in Last-In, First-Out (LIFO)"]
    Q3 --> A3["Use a Stack\nUndo, RNA secondary structure, parsing,
reversal"]

    Start --> Q4["Processing in First-In, First-Out (FIFO)"]
    Q4 --> A4["Use a Circular Queue or Linked Queue\nStreaming jobs,
rolling windows"]

    Start --> Q5["Always Extract Highest / Lowest Priority Item"]
    Q5 --> A5["Use a Priority Queue (Heap)\nO(log n) extract, O(1) peek"]

    Start --> Q6["Sort Data Fast with Guaranteed Ceiling"]
    Q6 --> A6["Use Merge Sort\nTheta(n log n) guaranteed, stable"]
```

---

4. Midterm Tracing Templates

When asked to trace an algorithm step-by-step on an exam, use these clean formats on your exam paper:

Tracing Selection Sort (Min-to-Front)

Pass i	Unsorted Region	Index of Min	Swap Operation	Array State After Pass
0	`[ 5, 2, 4, 1 ]`	3 (val: 1)	Swap `arr[0]` (5) with `arr[3]` (1)	`[ 1 | 2, 4, 5 ]`
1	`[ 2, 4, 5 ]`	1 (val: 2)	Swap `arr[1]` (2) with `arr[1]` (2)	`[ 1, 2 | 4, 5 ]`
2	`[ 4, 5 ]`	2 (val: 4)	Swap `arr[2]` (4) with `arr[2]` (4)	`[ 1, 2, 4 | 5 ]`

Tracing Insertion Sort

Pass i	`key`	Comparisons with Sorted Hand	Elements Shifted Right	Array State After Insertion
1	2	Compare with 5 ($5 > 2$)	5 shifts to index 1	`[ 2, 5 | 4, 1 ]`
2	4	Compare with 5 ($5 > 4$), compare with 2 ($2 < 4$)	5 shifts to index 2	`[ 2, 4, 5 | 1 ]`
3	1	Compare with 5, 4, 2 (all larger)	5, 4, 2 all shift right	`[ 1, 2, 4, 5 ]`

Tracing Circular Queue Operations

Operation	Value	`front`	`rear`	`count`	Array Contents `[0, 1, 2, 3]`
Initial	—	0	0	0	`[ _, _, _, _ ]`
`enqueue(10)`	10	0	1	1	`[ 10, _, _, _ ]`
`enqueue(20)`	20	0	2	2	`[ 10, 20, _, _ ]`
`dequeue()`	Returns 10	1	2	1	`[ _, 20, _, _ ]`
`enqueue(30)`	30	1	3	2	`[ _, 20, 30, _ ]`
`enqueue(40)`	40	1	0 (wrap!)	3	`[ _, 20, 30, 40 ]`
`enqueue(50)`	50	1	1 (wrap!)	4 (Full)	`[ 50, 20, 30, 40 ]`